

ระบบติดตามสินค้าด้วย RFID/GPS แบบเรียลไทม์

THE REAL-TIME PRODUCT TRACKING SYSTEM USING RFID/GPS

เสนอ

นาย ธีรดนย์ ทองเทพ 67200128

นาย วรรลภย์ ศรีเบญจรัตน์ 67200244

นาย วริทธิ์ธร คำภา 67200245

เสนอ

ดร. สุภาพร ชุ่มเพ็ญ

รายงานนี้เป็นส่วนหนึ่งของรายวิชา Internet of Things System Design

หลักสูตรวิศวกรรมศาสตร์บัณฑิต สาขาวิชาวิศวกรรมหุ่นยนต์และอิเล็กทรอนิกส์

อัจฉริยะ สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

วิทยาเขตชุมพรเขตอุดมศักดิ์จังหวัดชุมพร

ปีการศึกษา 2569

รายงานปีการศึกษา 2569

สาขาวิชาวิศวกรรมหุ่นยนต์และอิเล็กทรอนิกส์อัจฉริยะ

สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง วิทยาเขตชุมพรเขตอุดมศักดิ์ จังหวัดชุมพร

เรื่อง ระบบติดตามสินค้าด้วย RFID/GPS แบบเรียลไทม์

THE REAL-TIME PRODUCT TRACKING SYSTEM USING RFID/GPS

ผู้จัดทำ

1.นาย ธีรดนย์ ทองเทพ รหัสนักศึกษา 67200128

2.นายวรรลภย์ ศรีเบญจรัตน์ รหัสนักศึกษา 67200244

3.นายวริทธิ์ธร คำภา รหัสนักศึกษา 67200245

อาจารย์ที่ปรึกษา

(ดร. สุภาพร ชุ่มเพ็ญ)

(.....)

ชื่อรายงาน ระบบติดตามสินค้าด้วย RFID/GPS แบบเรียลไทม์
ชื่อผู้จัดทำ นายธีรดนย์ ทองเทพ รหัสนักศึกษา 67200128
นายวรรลภย์ ศรีเบญจรัตน์ รหัสนักศึกษา 67200244
นายวริทธิ์ธร คำภา รหัสนักศึกษา 67200245
อาจารย์ที่ปรึกษา ดร. สุภาพร ชุ่มเพ็ญ
หลักสูตร วิศวกรรมศาสตร์บัณฑิต
สาขาวิชา วิศวกรรมหุ่นยนต์และอิเล็กทรอนิกส์อัจฉริยะ
ปีการศึกษา 2569

บทคัดย่อ

โครงงานนี้นำเสนอการพัฒนาระบบติดตามสินค้าด้วยเทคโนโลยี RFID ร่วมกับ GPS แบบเรียลไทม์บนสถาปัตยกรรม IoT เพื่อป้องกันปัญหาสินค้าสูญหายระหว่างขนส่ง และเพิ่มประสิทธิภาพการบริหารจัดการระบบโลจิสติกส์

ระบบแบ่งออกเป็น 3 ส่วนหลัก ได้แก่

1. ส่วนฮาร์ดแวร์ปลายทาง (Edge Node): ใช้ไมโครคอนโทรลเลอร์ ESP32 ร่วมกับโมดูล RFID (MFRC522) สำหรับสแกนแท็กระบุตัวตนสินค้าและควบคุมการเปิด-ปิดรอบการติดตาม (Start/Stop Tracking) พร้อมแก้ปัญหาเสถียรภาพชิปด้วยระบบรีเซ็ตฮาร์ดแวร์ ทำงานร่วมกับโมดูล GPS เพื่ออ่านพิกัด ความเร็ว และคำนวณระยะทางสะสม
2. ส่วนส่งผ่านข้อมูล (Telemetry): แพ็กข้อมูล UID, พิกัด GPS, ความเร็ว, ระยะทาง และค่า RSSI ในรูปแบบ JSON ส่งผ่านโปรโตคอล MQTT เข้าสู่ EMQX Broker ทุกๆ 2 วินาที
3. ส่วนแสดงผล (Node-RED Dashboard): แยกประมวลผลข้อมูลออกเป็น 3 ทิศทางเพื่อแสดงผลบนแผนที่โต้ตอบ (Interactive Map), การ์ดข้อมูลพนักงาน และการ์ดสถานะการจัดส่งพัสดุแบบเรียลไทม์

ผลการทดสอบพบว่า ระบบสามารถระบุตัวตน สลับสภาวะการทำงาน และอัปเดตตำแหน่งพิกัดบน Node-RED Dashboard ได้อย่างแม่นยำและมีความหน่วงต่ำ สามารถนำไปประยุกต์ใช้งานในอุตสาหกรรมการขนส่งและคลังสินค้าได้จริง

Project Title The Real-Time Product Tracking System Using RFID/GPS

Author Name Teeradon Tongtap ID:67200128

Mr.Wanlop Sribenjarat ID:67200244

Mr.Warittorn Khampa ID:67200245

Advisor Dr. Supaporn Chumpen.

Degree Bachelor of Engineering.

Program in Robotics and Intelligent Electronics Engineering.

Academic Year 2026

ABSTRACT

This project develops an integrated Real-Time Product Tracking System using RFID and GPS technologies based on IoT architecture to prevent cargo loss and enhance logistics management efficiency.

The system comprises three core components:

1. Edge Node: Uses an ESP32 microcontroller with an RFID module (MFRC522) to scan item tags and toggle tracking sessions, incorporating a hardware reset mechanism to ensure stability. A GPS module simultaneously tracks real-time coordinates, speed, and cumulative distance.
2. Wireless Telemetry: Packages UID, GPS coordinates, speed, distance, and RSSI into JSON payloads, transmitted via MQTT protocol to an EMQX Broker every 2 seconds.
3. Node-RED Dashboard: Processes and routes data into three live displays: an interactive map, a staff information card, and a delivery status card.

Experimental results demonstrate that the system accurately identifies cargo, updates tracking coordinates with low latency on the Node-RED dashboard, and offers a practical solution for logistics and supply chain applications.

กิตติกรรมประกาศ

โครงงานเรื่อง "ระบบติดตามสินค้าด้วย RFID/GPS แบบเรียลไทม์"
(The Real-Time Product Tracking System Using RFID/GPS) สำเร็จลุล่วงไปได้ด้วยดีด้วยความกรุณาและความช่วยเหลืออย่างดียิ่งจาก [ยศ/ตำแหน่งทางวิชาการ ชื่อ-นามสกุล อาจารย์ที่ปรึกษา] อาจารย์ที่ปรึกษาโครงงาน ที่ได้ให้คำแนะนำ แนวคิด ตลอดจนช่วยแก้ไขปัญหาข้อบกพร่องต่าง ๆ ในด้านการพัฒนาระบบฮาร์ดแวร์ การเขียนโปรแกรมไมโครคอนโทรลเลอร์ และระบบเครือข่าย IoT มาโดยตลอด คณะผู้จัดทำขอขอบพระคุณเป็นอย่างสูงไว้ ณ โอกาสนี้

ขอขอบพระคุณคณาจารย์ประจำสาขาวิชาวิศวกรรมหุ่นยนต์และอิเล็กทรอนิกส์อัจฉริยะ สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง วิทยาเขตชุมพรเขตอุดมศักดิ์ ทุกท่าน ที่ได้สั่งสอนประสิทธิ์ประสาทวิชาความรู้ และบ่มเพาะทักษะทางวิศวกรรม อันเป็นพื้นฐานสำคัญในการนำมาประยุกต์ใช้จัดทำโครงงานฉบับนี้

ขอขอบพระคุณ บิดา มารดา และครอบครัว ที่ให้การสนับสนุนทั้งด้านทุนทรัพย์ กำลังใจ และความช่วยเหลือในทุก ๆ ด้าน รวมทั้งขอขอบคุณเพื่อน ๆ นักศึกษาทุกคนที่คอยให้คำแนะนำ แลกเปลี่ยนความรู้ และเป็นกำลังใจให้แก่คณะผู้จัดทำมาโดยตลอด

คุณค่าและประโยชน์อันเกิดจากโครงงานฉบับนี้ คณะผู้จัดทำขอมอบเป็นเครื่องบูชาพระคุณแก่ครูอาจารย์ บิดามารดา และผู้มีพระคุณทุกท่าน และหวังเป็นอย่างยิ่งว่าโครงงานนี้จะเป็นประโยชน์และเป็นแนวทางในการศึกษาค้นคว้าสำหรับการพัฒนาระบบติดตามพัสดุและเทคโนโลยีโลจิสติกส์ต่อไป

ธีรดนย์ ทองเทพ
วรรลภย์ ศรีเบญจรัตน์
วริทธิ์ธร คำภา

สารบัญ

เรื่อง	หน้า
บทคัดย่อภาษาไทย	I
บทคัดย่อภาษาอังกฤษ	II
กิตติกรรมประกาศ	III
สารบัญ	V-VII
สารบัญตาราง	VIII
สารบัญรูป	IX
บทที่ 1 บทนำ	1
1.1 ความเป็นมาและความสำคัญของปัญหา	1
1.2 วัตถุประสงค์ของโครงงาน	2
1.3 ขอบเขตของโครงงาน	2
1.3.1 ขอบเขตด้านฮาร์ดแวร์	2
1.3.2 ขอบเขตด้านซอฟต์แวร์และเฟิร์มแวร์	3
1.3.3 ขอบเขตด้านเครือข่ายและระบบแสดงผล	3
1.3.4 ขอบเขตการทดสอบและข้อจำกัดของระบบ	3
1.4 ประโยชน์ที่คาดว่าจะได้รับ	4
1.5 แผนการดำเนินงานและระยะเวลา	5
บทที่ 2 ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง	6
2.1 เทคโนโลยีการระบุตัวตนด้วยคลื่นความถี่วิทยุ	6
2.1.1 หลักการทำงานของ RFID Reader และ RFID Tag	8
2.2 ระบบระบุตำแหน่งบนพื้นโลกด้วยดาวเทียม	9
2.2.1 โครงสร้างข้อมูล NMEA Sentence และโมดูล u-blox NEO-M8N	9
2.2.2 การประมวลผลข้อมูลด้วยไลบรารี TinyGPS++	10

สารบัญ (ต่อ)

เรื่อง	หน้า
2.3 ไมโครคอนโทรลเลอร์ ESP32	11
2.3.1 คุณสมบัติและการจัดสรรพิน	12
2.3.2 การจัดการพลังงานและปัญหากระแสไฟตกขณะส่งสัญญาณ Wi-Fi	13
2.4 โปรโตคอลการสื่อสาร MQTT และระบบ EMQX Broker	15
2.4.1 สถาปัตยกรรมแบบ Publisher / Broker / Subscriber	15
2.4.2 โครงสร้างแพ็กเกจข้อมูลในรูปแบบ JSON	16
2.5 แพลตฟอร์ม Node-RED และ Node-RED Dashboard	18
2.5.1 การทำงานเชิงโหนด	19
2.5.2 การจัดการแผนที่โต้ตอบและการ์ดแสดงผล	19
บทที่ 3 การออกแบบและพัฒนาระบบ	21
3.1 ภาพรวมสถาปัตยกรรมของระบบ	21
3.2 การออกแบบส่วนฮาร์ดแวร์	23
3.2.1 การต่อวงจรและการจัดสรรพินสัญญาณ	24
3.2.2 ผังการเชื่อมต่อสายสัญญาณและระบบจ่ายไฟ	26
3.3 การพัฒนาเฟิร์มแวร์บนไมโครคอนโทรลเลอร์	30
3.3.1 อัลกอริทึมการสแกนบัตรสลับสภาวะการทำงาน	30
3.3.2 อัลกอริทึมการคำนวณระยะทางสะสมและความเร็วจากพิกัด GPS	32
3.3.3 การแก้ปัญหาชิป RFID ค้างด้วยฟังก์ชันรีเซ็ตฮาร์ดแวร์	33
3.3.4 การเชื่อมต่อเครือข่าย Wi-Fi และการส่งผ่านข้อมูล MQTT Payload	34
3.4 การพัฒนาส่วนแสดงผลบน Node-RED Dashboard	35
3.4.1 การตั้งค่าโหนดรับข้อมูลจาก EMQX Broker	36
3.4.2 การเขียน JavaScript ใน Function Node สำหรับกระจายข้อมูล 3 ส่วน	36
3.4.3 การออกแบบหน้าต่างแสดงผล Live Tracking Dashboard	39

สารบัญ (ต่อ)

เรื่อง	หน้า
บทที่ 4 ผลการทดลองและการวิเคราะห์ผล	41
4.1 การตั้งค่าและการเตรียมการทดสอบ	41
4.1.1 สภาพแวดล้อมในการทดสอบ	41
4.1.2 อุปกรณ์และเครื่องมือที่ใช้ในการทดสอบ	42
4.1.3 ขั้นตอนและวิธีการทดสอบระบบ	43
4.2 ผลการทดสอบการอ่านแท็ก RFID	44
4.2.1 ผลการทดสอบระยะการอ่านแท็ก RFID MFRC522	44
4.3 ผลการทดสอบความแม่นยำของพิกัด GPS และการคำนวณระยะทางสะสม	46
4.3.1 ผลการทดสอบความแม่นยำของพิกัดตำแหน่ง GPS	46
4.3.2 ผลการทดสอบการคำนวณระยะทางสะสม	47
4.4 ผลการทดสอบอัตราการสูญหายของแพ็กเกจข้อมูล	49
4.5 การวิเคราะห์และอภิปรายผลการทดลอง	50
4.5.1 ด้านประสิทธิภาพฮาร์ดแวร์และเสถียรภาพของระบบ	50
4.5.2 ด้านความแม่นยำของพิกัด GPS และการคำนวณระยะทาง	51
4.5.3 ด้านการสื่อสารข้อมูลและการแสดงผลบน Dashboard	51

สารบัญ (ต่อ)

เรื่อง	หน้า
บทที่ 5 บทสรุปและข้อเสนอแนะ	52
5.1 สรุปผลการดำเนินงาน	52
5.1.1 ด้านฮาร์ดแวร์และการลงทะเบียนด้วย RFID	52
5.1.2 ด้านการระบุพิกัด GPS และการคำนวณระยะทางสะสม	52
5.1.3 ด้านการสื่อสารข้อมูลไร้สายผ่าน MQTT	53
5.1.4 ด้านการแสดงผลและการโต้ตอบบน Live Tracking Dashboard	53
5.2 ปัญหา อุปสรรค และแนวทางการแก้ไข	54
5.3 ข้อเสนอแนะเพื่อการพัฒนาต่อยอด	55
5.3.1 ด้านการพัฒนาฮาร์ดแวร์และการจัดการพลังงาน	55
5.3.2 ด้านการสื่อสารข้อมูลและความปลอดภัยของระบบ	55
5.3.3 ด้านซอฟต์แวร์ การวิเคราะห์ข้อมูล และการแสดงผล	56
5.3.4 ด้านการผลิตและการใช้งานจริงเชิงอุตสาหกรรม	56
บรรณานุกรม	57
ภาคผนวก ก รหัสต้นฉบับการโปรแกรมไมโครคอนโทรลเลอร์ (ESP32 Source Code)	58
ภาคผนวก ข รหัสโครงสร้าง Node-RED Flow (JSON Export Node-RED)	61

สารบัญตาราง

ตารางที่	หน้า
1.5 แผนการดำเนินงานและระยะเวลา	5
3.2 การออกแบบส่วนฮาร์ดแวร์	23
3.1 สรุปการเชื่อมต่อพินสำหรับโหนดที่ 1	23
3.2 สรุปการเชื่อมต่อพินสำหรับโหนดที่ 2	26
4.1 อุปกรณ์และเครื่องมือที่ใช้ในการทดสอบ	42
4.2 ผลการทดสอบระยะการอ่านแท็ก RFID MFRC522	44
4.3.1 ผลการทดสอบความแม่นยำของพิกัดตำแหน่ง GPS	46
4.3.2 ผลการทดสอบการคำนวณระยะทางสะสม	48
4.4 ผลการทดสอบอัตราการสูญหายของแพ็กเกจข้อมูล	49

สารบัญรูป

รูปที่	หน้า
2.1.1 หลักการทำงานของ RFID Reader และ RFID Tag	6
2.1.2 การสื่อสารอนุกรมแบบ SPI	8
2.3.1 คุณสมบัติและการจัดสรรพิน	1
2.3.2 การจัดการพลังงานและปัญหากระแสไฟตกขณะส่งสัญญาณ Wi-Fi	13
2.4.1 สถาปัตยกรรมแบบ Publisher / Broker / Subscriber	15
2.5.2 การจัดการแผนที่โต้ตอบและการ์ดแสดงผล	20
3.1 ภาพรวมสถาปัตยกรรมของระบบ	21
1. ผังการเชื่อมต่อโหนดที่ 1: RFID Station Node	27
2. ผังการเชื่อมต่อโหนดที่ 2: Mobile GPS Tracker Node	29
1. โครงสร้างและการจัดวางส่วนประกอบบน Dashboard	39

บทที่ 1

บทนำ

1.1 ความเป็นมาและความสำคัญของปัญหา

ในปัจจุบัน อุตสาหกรรมขนส่งและโลจิสติกส์ (Logistics and Supply Chain) มีการเติบโตอย่างรวดเร็วตามการขยายตัวของการค้าปลีกออนไลน์ (E-Commerce) และระบบการค้าโลก การบริหารจัดการห่วงโซ่อุปทานที่มีประสิทธิภาพจึงกลายเป็นหัวใจสำคัญในการสร้างความได้เปรียบทางธุรกิจ ซึ่งหัวใจหลักของการดำเนินงานด้านโลจิสติกส์ไม่ได้อยู่เพียงแค่การส่งมอบสินค้าให้ถึงปลายทางเท่านั้น แต่ยังรวมถึงความแม่นยำ ความปลอดภัย และความสามารถในการตรวจสอบสถานะของสินค้าได้ตลอดกระบวนการจัดส่ง

อย่างไรก็ตาม ปัญหาที่มักพบในกระบวนการขนส่งรูปแบบเดิม คือการสูญหายของพัสดุระหว่างทาง การระบุตัวตนสินค้าผิดพลาด และการขาดระบบติดตามตำแหน่งที่มีความแม่นยำแบบเรียลไทม์ (Real-Time Monitoring) ระบบติดตามสินค้าส่วนใหญ่ในปัจจุบันยังคงพึ่งพาการบันทึกข้อมูลแบบดั้งเดิมหรือการสแกนบาร์โค้ดด้วยมนุษย์เฉพาะ ณ จุดพักสินค้า ซึ่งไม่สามารถระบุตำแหน่งที่แท้จริงของยานพาหนะขณะขนส่งได้ ในทางกลับกัน หากใช้เพียงระบบติดตามตำแหน่งด้วย GPS เพียงอย่างเดียว ก็จะไม่สามารถยืนยันตัวตนสินค้าแบบรายชิ้น หรือตรวจสอบสภาวะการเริ่มและสิ้นสุดการจัดส่งของพนักงานได้อย่างแม่นยำ

จากปัญหาดังกล่าว การนำเทคโนโลยีอินเทอร์เน็ตของสรรพสิ่ง (Internet of Things: IoT) มาประยุกต์ใช้ร่วมกับเทคโนโลยีการระบุตัวตนด้วยคลื่นความถี่วิทยุ (RFID) และระบบระบุตำแหน่งบนพื้นโลก (GPS) จึงเป็นแนวทางที่สามารถแก้ช่องว่างนี้ได้อย่างมีประสิทธิภาพ โดยเทคโนโลยี RFID (MFRC522) นำมาใช้ระบุตัวตนและตรวจสอบสิทธิ์พนักงานหรือรหัสสินค้า ณ จุดลงทะเบียนเข้า-ออกคลัง พร้อมใช้เป็นเงื่อนไขสวิตช์ในการเริ่มหรือจบสภาวะการติดตาม (Start/Stop Tracking) ขณะที่โมดูล GPS นำมาใช้ดึงพิกัดทางภูมิศาสตร์ คำนวณความเร็ว และคำนวณระยะทางสะสมของยานพาหนะระหว่างการเดินทาง

นอกจากนี้ การส่งผ่านข้อมูลในระบบติดตามแบบเรียลไทม์จำเป็นต้องมีความหน่วงต่ำ (Low Latency) และมีความเสถียรสูง โครงงานนี้จึงเลือกใช้ไมโครคอนโทรลเลอร์ ESP32 ทำหน้าที่เป็นหน่วยประมวลผลปลายทาง (Edge Node) ในการรวบรวมข้อมูลสถานะและพิกัด GPS แพ็กเป็นชุดข้อมูล JSON แล้วส่งผ่านโปรโตคอล MQTT เข้าสู่ EMQX Broker ไปยังแพลตฟอร์ม Node-RED Dashboard ซึ่งประมวลผลแยกข้อมูลออกเป็น 3 ส่วน ได้แก่ แผนที่โต้ตอบ (Interactive Map) การ์ดข้อมูลพนักงาน และการ์ดสถานะการจัดส่งพัสดุ พร้อมทั้งได้รับการออกแบบการจัดการพลังงานโดยมีอัลกอริทึมบังคับรีบูตชิป RFID (Hardware Force Reset) เพื่อแก้ปัญหาอุปกรณ์หยุดทำงานชั่วคราวขณะเสาอากาศ Wi-Fi ดึงกระแสไฟฟ้าส่งข้อมูล

ดังนั้น คณะผู้จัดทำจึงได้พัฒนาระบบติดตามสินค้าด้วย RFID/GPS แบบเรียลไทม์ ขึ้น เพื่อให้ได้ระบบติดตามที่มีความแม่นยำ เสถียรภาพสูง และสามารถแสดงผลข้อมูลตำแหน่งพร้อมสถานะสินค้าได้แบบเรียลไทม์ ซึ่งจะช่วยลดความเสี่ยงจากการสูญหาย เพิ่มความน่าเชื่อถือในกระบวนการจัดส่ง และสามารถนำไปประยุกต์ใช้งานในอุตสาหกรรมขนส่งและคลังสินค้าได้อย่างมีประสิทธิภาพ

1.2 วัตถุประสงค์ของโครงงาน

1. เพื่อออกแบบและสร้างชุดอุปกรณ์ฮาร์ดแวร์ติดตามสินค้าปลายทาง (Edge Node) ที่รวมการทำงานของโมดูล RFID (MFRC522) สำหรับสแกนระบุตัวตนและควบคุมสภาวะการทำงาน ร่วมกับโมดูล GPS สำหรับดึงพิกัดตำแหน่งทางภูมิศาสตร์ โดยใช้ไมโครคอนโทรลเลอร์ ESP32 เป็นหน่วยประมวลผลหลัก
2. เพื่อพัฒนาระบบรับ-ส่งข้อมูลไร้สายความหน่วงต่ำผ่านโปรโตคอล MQTT ร่วมกับ EMQX Broker พร้อมทั้งพัฒนาระบบจัดการพลังงานและอัลกอริทึมรีเซ็ตฮาร์ดแวร์ (Hardware Force Reset) เพื่อเพิ่มเสถียรภาพการทำงานของชิป RFID
3. เพื่อพัฒนาส่วนแสดงผลและบริหารจัดการข้อมูลบน Node-RED Dashboard สำหรับแสดงตำแหน่งยานพาหนะบนแผนที่แบบโต้ตอบ (Interactive Map) ข้อมูลพนักงานผู้จัดส่ง และสถานะการจัดส่งพัสดุแบบเรียลไทม์
4. เพื่อทดสอบและประเมินประสิทธิภาพการทำงานของระบบ ในด้านความแม่นยำของการอ่านแท็ก RFID, ความแม่นยำของพิกัด GPS, การคำนวณระยะทางสะสม และความหน่วงเวลา (Latency) ในการรับ-ส่งข้อมูล

1.3 ขอบเขตของโครงงาน

1.3.1 ขอบเขตด้านฮาร์ดแวร์

1. ไมโครคอนโทรลเลอร์หลัก: ใช้บอร์ด ESP32 เป็นหน่วยประมวลผลหลัก ทำหน้าที่ควบคุมโมดูลเชื่อมต่อ รวบรวมข้อมูล และส่งผ่านข้อมูลไร้สาย
2. ระบบระบุตัวตนสินค้าและพนักงาน: ใช้โมดูล RFID MFRC522 (ย่านความถี่ 13.56 MHz) เชื่อมต่อผ่านบัส SPI (พิน RST: 22, SS: 5) สำหรับสแกนบัตรหรือแท็ก RFID ในระยะอ่านไม่เกิน 5 เซนติเมตร โดยกำหนดให้การสแกนบัตรเป็นสวิตช์เปิด-ปิดสภาวะการติดตาม (Start/Stop Tracking)
3. ระบบระบุตำแหน่ง: ใช้โมดูลรับสัญญาณดาวเทียม GPS เชื่อมต่อผ่านการสื่อสารอนุกรม HardwareSerial2 (RX: Pin 32, TX: Pin 33) ที่ความเร็วบอร์ดเรต (Baud rate) 9600 bps
4. ระบบสื่อสารไร้สาย: เชื่อมต่ออินเทอร์เน็ตผ่านสัญญาณ Wi-Fi ย่านความถี่ 2.4 GHz

1.3.2 ขอบเขตด้านซอฟต์แวร์และเฟิร์มแวร์

1. การพัฒนาเฟิร์มแวร์: พัฒนาโปรแกรมบนไมโครคอนโทรลเลอร์ ESP32 ด้วยภาษา C/C++ บน Arduino IDE
2. การประมวลผลข้อมูล GPS: ใช้ไลบรารี TinyGPS++ ในการแกะรหัสชุดข้อมูล NMEA เพื่อแปลงเป็นพิกัดละติจูด (Latitude), ลองจิจูด (Longitude), ความเร็วเดินทาง (กิโลเมตร/ชั่วโมง) และคำนวณระยะทางสะสม (Total Distance)
3. การจัดการเสถียรภาพฮาร์ดแวร์: พัฒนาอัลกอริทึมบังคับรีบูตชิป RFID (Hardware Force Reset) ผ่านพิน RST เพื่อป้องกันปัญหาชิป RFID ค้างหรือหยุดทำงานจากกระแสไฟฟ้าตกขณะเสาอากาศ Wi-Fi ดึงกระแสไปส่งข้อมูล
4. รูปแบบโครงสร้างข้อมูล (Data Payload): แพ็กข้อมูลส่งออกในรูปแบบ JSON ประกอบด้วย รหัสแท็ก RFID (uid), พิกัดละติจูด (lat), พิกัดลองจิจูด (lng), ความเร็ว (speed), ระยะทางสะสม (dist), ค่าความเข้มสัญญาณ Wi-Fi (rssi) และสถานะการทำงาน (status)

1.3.3 ขอบเขตด้านเครือข่ายและระบบแสดงผล

1. โปรโตคอลรับ-ส่งข้อมูล: ใช้โปรโตคอล MQTT สื่อสารข้อมูลผ่านพอร์ต 1883 ไปยัง Public MQTT Broker (broker.emqx.io) โดยกำหนดหัวข้อการสื่อสาร (Topic) เป็น myname9988/gpsdata
2. รอบการส่งข้อมูล: กำหนดช่วงเวลาการส่งข้อมูล (Send Interval) ทุกๆ 2 วินาที ในขณะที่ระบบอยู่ในสภาวะกำลังติดตาม (isTracking = true)
3. แพลตฟอร์มแสดงผล (Dashboard): พัฒนาส่วนผู้ใช้งานบนแพลตฟอร์ม Node-RED Dashboard ในรูปแบบเว็บแอปพลิเคชัน โดยแบ่งส่วนแสดงผลออกเป็น 3 ส่วนหลัก ได้แก่
    - แผนที่โต้ตอบ (Interactive Map): แสดงตำแหน่งพิกัดของยานพาหนะขนส่งแบบเรียลไทม์
    - การ์ดข้อมูลพนักงาน (Staff Card): แสดงรหัสแท็ก RFID และสถานะการปฏิบัติงาน
    - การ์ดสถานะการจัดส่ง (Delivery Status Card): แสดงข้อมูลความเร็ว, ระยะทางสะสม, พิกัดตำแหน่ง, ความเข้มสัญญาณ Wi-Fi (RSSI) และเวลาที่มีการอัปเดตข้อมูลล่าสุด

1.3.4 ขอบเขตการทดสอบและข้อจำกัดของระบบ

1. การระบุตำแหน่งทางภูมิศาสตร์ครอบคลุมเฉพาะพื้นที่ที่โมดูล GPS สามารถรับสัญญาณดาวเทียมได้ (เปิดว่าง หรือมีสิ่งกีดขวางปานกลาง)
2. การส่งผ่านข้อมูลแบบเรียลไทม์จำกัดเฉพาะพื้นที่ที่มีสัญญาณการเชื่อมต่อ Wi-Fi 2.4 GHz
3. การคำนวณระยะทางสะสมเป็นการคำนวณระยะทางตรงระหว่างจุดพิกัด GPS สองจุดติดต่อกัน (Haversine Formula) บนไลบรารี TinyGPS++

1.4 ประโยชน์ที่คาดว่าจะได้รับ

การพัฒนาระบบติดตามสินค้าด้วย RFID และ GPS แบบเรียลไทม์ ร่วมกับ Node-RED และ ESP32 คาดว่าจะก่อให้เกิดประโยชน์ทั้งในเชิงวิศวกรรม IoT และเชิงการประยุกต์ใช้งานจริงในอุตสาหกรรมโลจิสติกส์ ดังนี้

1. ด้านการเพิ่มประสิทธิภาพการบริหารจัดการขนส่งและติดตามสินค้า (Logistics & Supply Chain)

- การติดตามตำแหน่งสินค้าแบบเรียลไทม์: ผู้ดูแลระบบสามารถตรวจสอบพิกัดตำแหน่งทางภูมิศาสตร์ ความเร็ว และระยะทางสะสมของยานพาหนะขนส่งผ่านแผนที่โต้ตอบ (Worldmap UI) บน Node-RED Dashboard ได้อย่างแม่นยำและต่อเนื่อง
- การควบคุมสภาวะจัดส่งที่เป็นระบบ: ระบบรองรับการสลับสภาวะเริ่มติดตาม (Start Tracking) และหยุดติดตาม (Stop Tracking) อัตโนมัติด้วยการสแกนแท็ก RFID ช่วยลดความผิดพลาดจากกระบวนการบันทึกข้อมูลด้วยมนุษย์ (Manual Data Entry)
- การเชื่อมโยงข้อมูลพนักงานและพัสดุ: สามารถจับคู่รหัสประจำตัวพนักงานขนส่ง (UID) เข้ากับสถานะการจัดส่งได้อย่างชัดเจน ทำให้การตรวจสอบย้อนกลับ (Traceability) มีความโปร่งใสและน่าเชื่อถือ

2. ด้านการออกแบบและพัฒนาฮาร์ดแวร์ IoT (Hardware & Firmware Engineering)

- โครงสร้างการทำงานแบบโหนดอิสระ (Dual-Node Architecture): ได้แนวทางการออกแบบที่แยกฟังก์ชันการลงทะเบียนประจำที่ (RFID Station) และการติดตามเคลื่อนที่ (Mobile GPS Tracker) ออกจากกัน ช่วยลดความซับซ้อนของสายสัญญาณและเพิ่มความคล่องตัวในการติดตั้งใช้งานจริง
- การแก้ปัญหาเสถียรภาพระบบฮาร์ดแวร์: ได้อัลกอริทึม forceResetRFID() ที่สามารถป้องกันและแก้ไขปัญหาชิป RFID MFRC522 ค้างจากภาวะแรงดันไฟตกชั่วขณะ (Voltage Dip) เมื่อ ESP32 ส่งข้อมูล Wi-Fi/MQTT ช่วยให้ระบบทำงานได้ต่อเนื่องโดยไม่ต้องรีบูตเครื่อง
- ระบบจัดการพลังงานภาคสนาม: ได้โครงสร้างวงจรจ่ายไฟเคลื่อนที่ที่เสถียร โดยการใช้แบตเตอรี่ Li-ion 18650 ร่วมกับโมดูล DC-DC Buck Converter HW-263 เพื่อแปลงแรงดันไฟเป็น **5.0V DC** สำหรับเลี้ยงไมโครคอนโทรลเลอร์ ESP32 ได้อย่างสม่ำเสมอ

3. ด้านความคุ้มค่าและต้นทุนการดำเนินงาน (Cost Efficiency & Scalability)

- ลดต้นทุนการพัฒนาระบบติดตาม: ใช้ฮาร์ดแวร์และอุปกรณ์ที่มีราคาเข้าถึงง่าย เช่น ไมโครคอนโทรลเลอร์ ESP32 และโมดูล GPS/RFID ร่วมกับซอฟต์แวร์โอเพ่นซอร์ส (Node-RED, EMQX MQTT Broker) ช่วยลดต้นทุนในการสร้างระบบต้นแบบเมื่อเทียบกับอุปกรณ์ GPS Tracker สำเร็จรูปในท้องตลาด
- รองรับการขยายระบบในอนาคต (Scalability): โครงสร้างการส่งข้อมูลแบบ JSON Payload บนโปรโตคอล MQTT ช่วยให้สามารถเพิ่มจำนวนโหนดติดตามสินค้า หรือนำข้อมูลไปประมวลผลต่อยอดร่วมกับระบบฐานข้อมูลและแจ้งเตือนภายนอกได้อย่างสะดวก

1.5 แผนการดำเนินงานและระยะเวลา

การดำเนินงานพัฒนาระบบติดตามสินค้าด้วย RFID และ GPS แบบเรียลไทม์ มีระยะเวลาการดำเนินโครงการรวมทั้งสิ้น 4 เดือน (16 สัปดาห์) โดยแบ่งขั้นตอนและกิจกรรมการดำเนินงานออกเป็นช่วงต่างๆ ดังแสดงในตารางแผนงาน (Gantt Chart) ต่อไปนี้

ขั้นตอนการดำเนินงาน	ระยะเวลา(สัปดาห์ที่)							
	1	2	3	4	5	6	7	8
1.ศึกษาค้นคว้าหาข้อมูล	↔							
2.จัดหาอุปกรณ์		↔						
3.ออกแบบและทดลอง			↔					
4.แก้ไขปัญหา				↔				
5.ประกอบโครงสร้าง				↔				
6.ทดสอบการทำงาน					↔			
7.สรุปผลและส่งงาน							↔	

1.5 แผนการดำเนินงานและระยะเวลา

บทที่ 2

ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง

2.1 เทคโนโลยีการระบุตัวตนด้วยคลื่นความถี่วิทยุ

เทคโนโลยีการระบุตัวตนด้วยคลื่นความถี่วิทยุ (Radio Frequency Identification: RFID) เป็นระบบสื่อสารไร้สายแบบไม่สัมผัส (Non-contact) ที่ใช้คลื่นแม่เหล็กไฟฟ้าในการรับ-ส่งข้อมูลระหว่างอุปกรณ์อ่านข้อมูล (Reader) และป้ายระบุข้อมูล (Tag) เพื่อใช้วิเคราะห์และยืนยันตัวตนของวัตถุ บุคคล หรือพัสดุได้อย่างแม่นยำและรวดเร็ว

2.1.1 หลักการทำงานของ RFID Reader และ RFID Tag

ระบบ RFID โดยทั่วไปประกอบด้วยองค์ประกอบหลัก 2 ส่วน ได้แก่ อุปกรณ์อ่าน-เขียนข้อมูล (RFID Reader/Writer) และป้ายอิเล็กทรอนิกส์ (RFID Tag)

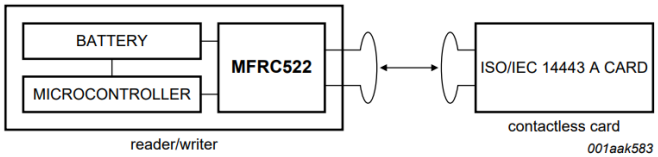
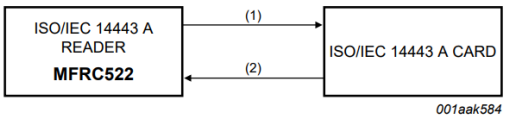


Fig 4. MFRC522 Read/Write mode

The physical level communication is shown in Figure 5.



(1) Reader to card 100 % ASK, Miller encoded, transfer speed 106 kBd to 848 kBd.
(2) Card to reader subcarrier load modulation, Manchester encoded or BPSK, transfer speed 106 kBd to 848 kBd.

Fig 5. ISO/IEC 14443 A/MIFARE Read/Write mode communication diagram

2.1.1 หลักการทำงานของ RFID Reader และ RFID Tag

1.โครงสร้างและหลักการกำเนิดพลังงาน (Inductive Coupling):

โครงงานนี้เลือกใช้ไอซี MFRC522 ซึ่งทำงานที่ย่านความถี่สูง (High Frequency: HF) 13.56 MHz ตามมาตรฐาน ISO/IEC 14443A ร่วมกับแท็กแบบไม่มีแหล่งจ่ายไฟในตัวเอง (Passive Tag)

- สายอากาศของ Reader จะปล่อยสนามแม่เหล็กความถี่วิทยุออกมาอย่างต่อเนื่อง
- เมื่อ Passive Tag เคลื่อนที่เข้ามาในระยะสนามแม่เหล็ก (ประมาณ 0–5 เซนติเมตร) คอยล์สายอากาศภายในแท็กจะเหนี่ยวนำแรงดันไฟฟ้ากระแสสลับตามกฎการเหนี่ยวนำของฟาราเดย์ (Faraday's Law of Induction)
- แท็กจะแปลงแรงดันไฟฟ้านี้เป็นไฟฟ้ากระแสตรงเพื่อป้อนพลังงานให้กับไมโครชิปภายในแท็ก
- ไมโครชิปจะทำการสะท้อนสัญญาณกลับ (Load Modulation) โดยส่งรหัสประจำตัวเฉพาะ (Unique Identifier: UID) ขนาด 4 ไบต์ หรือ 7 ไบต์ กลับมายัง Reader

2. คุณสมบัติสำคัญของชิป MFRC522:

- แรงดันไฟฟ้าใช้งาน: 3.3V ซึ่งสอดคล้องกับระดับแรงดันของไมโครคอนโทรลเลอร์ ESP32 โดยตรง
- บัฟเฟอร์รับ-ส่งข้อมูล: มี FIFO Buffer ขนาด 64 ไบต์ สำหรับกักเก็บข้อมูลการสื่อสาร
- พินรีเซ็ตฮาร์ดแวร์ (RST Pin): มีพินควบคุมการรีเซ็ตและตัดการทำงาน (Power-down Mode) ซึ่งสอดคล้องกับอัลกอริทึม forceResetRFID() ในโครงงานนี้ ที่ใช้ดึงสัญญาณพิน RST ลง LOW และ HIGH เพื่อล้างสภาวะการค้างของชิป MFRC522 หลังจากการส่งข้อมูลผ่านเครือข่าย Wi-Fi

2.1.2 การสื่อสารอนุกรมแบบ SPI

ไอซี MFRC522 เชื่อมต่อและแลกเปลี่ยนข้อมูลกับไมโครคอนโทรลเลอร์ ESP32 ผ่านการสื่อสารอนุกรมแบบซิงโครนัสความเร็วสูงชนิด SPI (Serial Peripheral Interface) ซึ่งทำงานในสถาปัตยกรรมแบบ Master-Slave (ESP32 ทำหน้าที่เป็น Master และ MFRC522 ทำหน้าที่เป็น Slave)

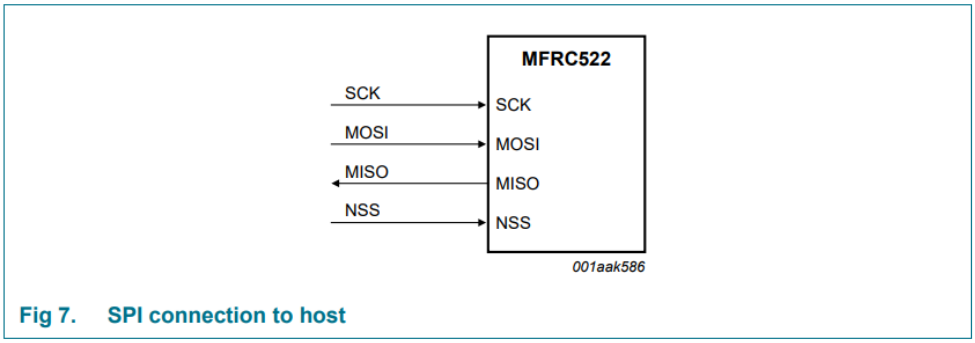


Fig 7. SPI connection to host

2.1.2 การสื่อสารอนุกรมแบบ SPI

การสื่อสารผ่านบัส SPI ประกอบด้วยสายสัญญาณหลัก 4 เส้น และสายควบคุมเสริมดังนี้:

1. SCK / SCLK (Serial Clock): สัญญาณนาฬิกากำหนดจังหวะการรับ-ส่งข้อมูล สร้างขึ้นโดย ESP32 (Master)
2. MOSI (Master Out Slave In): สายส่งข้อมูลออกจาก ESP32 เข้าสู่โมดูล MFRC522
3. MISO (Master In Slave Out): สายส่งข้อมูลออกจาก MFRC522 กลับไปยัง ESP32
4. SS / SDA / CS (Slave Select / Chip Select): สัญญาณเลือกอุปกรณ์ Slave ทำงานแบบ Active Low (ในโครงงานนี้ต่อเข้ากับพิน GPIO5 ของ ESP32) เมื่อ ESP32 ต้องการดึงข้อมูล UID จาก RFID จะส่งสัญญาณ LOW ไปยังพินนี้
5. RST (Reset): สายสัญญาณรีเซ็ตฮาร์ดแวร์ (ต่อเข้ากับพิน GPIO22 ของ ESP32) ทำหน้าที่รีบูตและเริ่มต้นกระบวนการสื่อสารของชิป MFRC522 ใหม่เมื่อระบบต้องการคืนค่าสภาวะเริ่มต้นการทำงาน

2.2 ระบบระบุตำแหน่งบนพื้นโลกด้วยดาวเทียม

ระบบระบุตำแหน่งบนพื้นโลกด้วยดาวเทียม (Global Positioning System: GPS) เป็นส่วนหนึ่งของระบบดาวเทียมนำทางสากล (Global Navigation Satellite System: GNSS) ที่ใช้ระบุพิกัดทางภูมิศาสตร์ของผู้รับสัญญาณบนผิวโลก โดยอาศัยการคำนวณระยะห่างจากดาวเทียมนำทางที่โคจรรอบโลกด้วยวิธีทรงกลมเฉือนกัน (Trilateration)

โครงงานนี้เลือกใช้โมดูลรับสัญญาณ u-blox NEO-M8N ซึ่งเป็นโมดูล GNSS ประสิทธิภาพสูง ยุคใหม่ที่รองรับการรับสัญญาณจากกลุ่มดาวเทียมหลายระบบพร้อมกัน (Concurrent GNSS) เช่น GPS (สหรัฐอเมริกา), GLONASS (รัสเซีย), BeiDou (จีน) และ Galileo (สหภาพยุโรป) ทำให้มีความไวในการดึงสัญญาณ (Tracking Sensitivity) สูงถึง -167 dBm ระยะเวลาในการเริ่มล็อกสัญญาณพิกัดครั้งแรก (Time-To-First-Fix: TTFF) รวดเร็ว และมีความแม่นยำของพิกัดสูงกว่าโมดูล GPS รุ่นเก่า เหมาะสำหรับการติดตามยานพาหนะขนส่งในสภาวะที่มีสิ่งกีดขวางหรือในพื้นที่เมือง

2.2.1 โครงสร้างข้อมูล NMEA Sentence และโมดูล u-blox NEO-M8N

โมดูล u-blox NEO-M8N สื่อสารและส่งข้อมูลพิกัดออกมาทางพอร์ตการสื่อสารอนุกรม (UART) ที่ความเร็วบอร์ดเรต 9600 bps ในรูปแบบข้อความมาตรฐาน NMEA 0183
(National Marine Electronics Association)

เนื่องจากโมดูล NEO-M8N รองรับระบบ Multi-GNSS รหัสประจำประโยคข้อมูลจึงขึ้นต้นด้วย $GN (Global Navigation) หรือ $GP (GPS) โดยประโยคข้อมูลหลักที่ใช้ในการติดตามตำแหน่งยานพาหนะคือ $GNRMC (Recommended Minimum Specific GNSS Data) ซึ่งมีตัวอย่างชุดข้อมูลดังนี้:

GNRMC,123519.00,A,1345.1234,N,10030.5678,E,0.02,120.5,250926,,,A*6A

โครงสร้างฟิลด์ข้อมูลภายในประโยค $GNRMC ประกอบด้วย:

1. $GNRMC: รหัสระบุประเภทประโยคข้อมูลระบุตำแหน่งขั้นต่ำจากระบบดาวเทียมรวม (Multi-GNSS)
2. 123519.00: เวลา UTC ในรูปแบบ hhmmss.ss (12:35:19.00 น.)
3. A: สภาวะการรับสัญญาณ (A = Valid / สัญญาณปกติพร้อมใช้งาน, V = Void / สัญญาณไม่สมบูรณ์)
4. 1345.1234,N: พิกัดละติจูด 13 เหนือ (North)
5. 10030.5678,E: พิกัดลองจิจูด 100 ตะวันออก (East)
6. 0.02: ความเร็วในการเคลื่อนที่ขณะนั้น มีหน่วยเป็น นอต (Knots)
7. 120.5: ทิศทางการเดินทาง (Course Over Ground) มีหน่วยเป็นองศา (0–359.9)
8. 250926: วัน เดือน ปี UTC ในรูปแบบ ddmmyy (25 กันยายน ค.ศ. 2026)
9. *6A: ค่า Checksum สำหรับตรวจทานความถูกต้องของข้อมูล

2.2.2 การประมวลผลข้อมูลด้วยไลบรารี TinyGPS++

การแกะรหัสสตริง NMEA สตรีมมิ่งจากโมดูล NEO-M8N กระทำผ่านไมโครคอนโทรลเลอร์ ESP32 โดยใช้ไลบรารี TinyGPS++ เพื่อถอดรหัสและดึงค่าพารามิเตอร์มาประมวลผลต่อในเฟิร์มแวร์

1. การอ่านข้อมูลผ่านการสื่อสารอนุกรม:
   ESP32 เชื่อมต่อกับโมดูล NEO-M8N ผ่าน HardwareSerial2 (พิน RX: 32, TX: 33)
   และนำไบต์ข้อมูลเข้าสู่ไลบรารีด้วยคำสั่ง:

C++

```
while (gpsSerial.available() > 0)
    {
     gps.encode(gpsSerial.read());
    }
```

2. การจัดการพารามิเตอร์ตำแหน่งและความเร็ว:
   - gps.location.isValid(): ตรวจสอบสถานะการล็อกสัญญาณดาวเทียมของโมดูล NEO-M8N ก่อนนำพิกัดไปคำนวณ
   - gps.location.lat() และ gps.location.lng(): ดึงค่าพิกัดละติจูดและลองจิจูดในรูปแบบทศนิยม (Decimal Degrees) เพื่อจัดใส่โครงสร้าง JSON
   - gps.speed.kmph(): อ่านค่าความเร็วและแปลงหน่วยเป็นกิโลเมตรต่อชั่วโมง (km/h)

3. การคำนวณระยะทางสะสม (Distance Calculation):

การคำนวณระยะทางระหว่างพิกัดเดิม (lat_1, lng_1) และพิกัดใหม่ (lat_2, lng_2) คำนวณด้วยสูตรไฮเวอร์ซีน (Haversine Formula) ซึ่งคำนึงถึงความโค้งของผิวโลก:

$$d = 2R \cdot \arcsin\left(\sqrt{\sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1) \cdot \cos(\phi_2) \cdot \sin^2\left(\frac{\Delta\lambda}{2}\right)}\right)$$

โดยที่ ϕ_1, ϕ_2 คือพิกัดละติจูดในหน่วยเรเดียน, $\Delta\phi = \phi_2 - \phi_1$, $\Delta\lambda = \lambda_2 - \lambda_1$ และ R คือรัศมีเฉลี่ยของโลก ($R \approx 6{,}371$ กิโลเมตร)

ในโค้ดระบบจะเรียกใช้คำสั่งของไลบรารีเพื่อคำนวณระยะทางย่อยระหว่างจุดและนำไปสะสมในตัวแปร totalDistance:

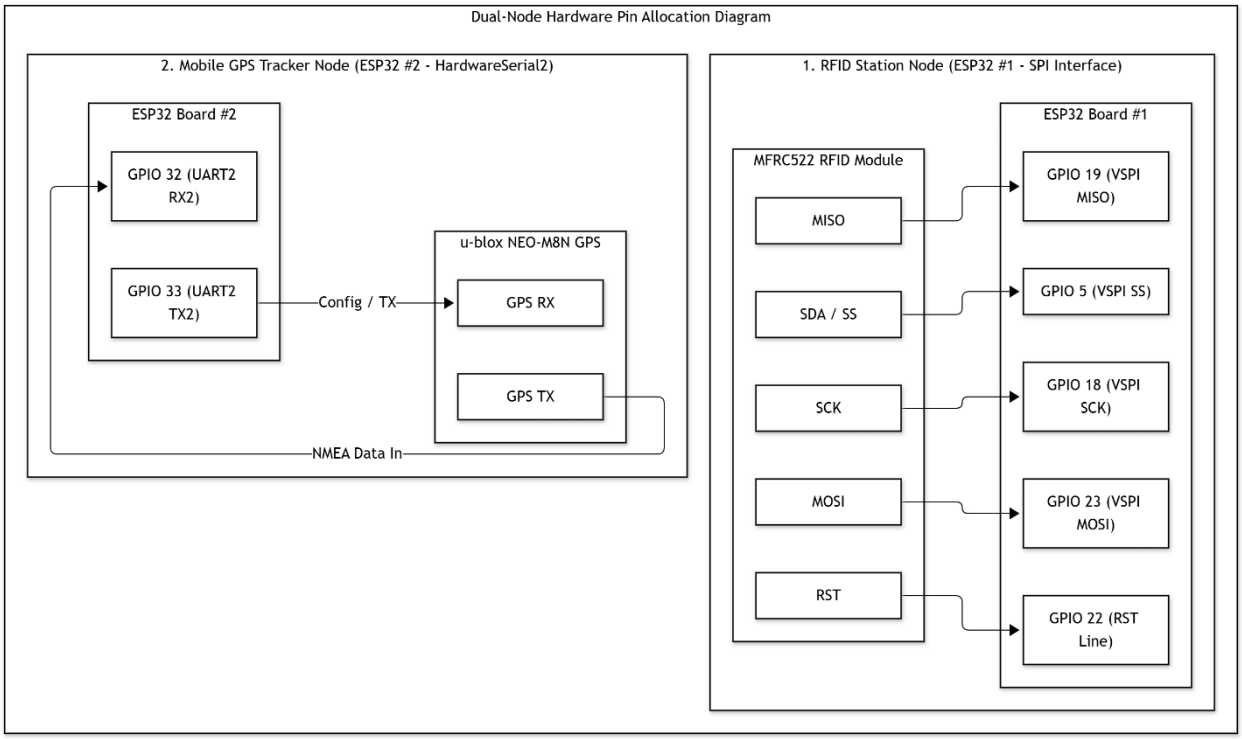
```
totalDistance += TinyGPSPlus::distanceBetween(previousLat, previousLng, lat, lng);
```

2.3 ไมโครคอนโทรลเลอร์ ESP32

ไมโครคอนโทรลเลอร์ ESP32 เป็นชิปประมวลผลตระกูล 32-บิต พัฒนาโดยบริษัท Espressif Systems ใช้สถาปัตยกรรม Xtensa® Dual-Core 32-bit LX6 มีความเร็วในการประมวลผลสูงสุด 240 MHz มีจุดเด่นคือการรวมโมดูลการสื่อสารไร้สาย Wi-Fi (มาตรฐาน 802.11 b/g/n ย่านความถี่ 2.4 GHz) และ Bluetooth
(v4.2 BR/EDR และ BLE) ไว้ภายในชิปตัวเดียว จึงเหมาะสำหรับการนำมาเป็นหน่วยประมวลผลหลักปลายทาง (Edge Node) ในโครงงานระบบติดตามสินค้าที่ต้องการประมวลผลข้อมูลเซนเซอร์พร้อมกันและส่งผ่านข้อมูลขึ้นระบบคลาวด์แบบเรียลไทม์

2.3.1 คุณสมบัติและการจัดสรรพิน

ไมโครคอนโทรลเลอร์ ESP32 มีพินต่อใช้งานภายนอก (General Purpose Input/Output: GPIO) ที่รองรับการกำหนดหน้าที่ผ่านซอฟต์แวร์ (Multiplexing) ในโครงงานนี้ได้ออกแบบและจัดสรรพินสัญญาณเพื่อเชื่อมต่อกับอุปกรณ์ต่าง ๆ ดังนี้:



2.3.1 คุณสมบัติและการจัดสรรพิน

1. การจัดสรรพินสำหรับ ESP32 ตัวที่ 1 (RFID Station Node)

ไมโครคอนโทรลเลอร์ ESP32 ตัวที่ 1 ทำหน้าที่เป็นสถานีสแกนประจำที่ เชื่อมต่อกับโมดูล RFID MFRC522 ผ่านบัสการสื่อสารอนุกรมความเร็วสูง VSPI (Virtual Serial Peripheral Interface) โดยจัดสรรพินดังนี้:

- พินสัญญาณบัส SPI หลัก (SCK / MISO / MOSI):
    - SCK (Serial Clock): กำหนดใช้ GPIO 18 ทำหน้าที่ส่งสัญญาณนาฬิกาควบคู่การรับ-ส่งข้อมูล
    - MISO (Master In Slave Out): กำหนดใช้ GPIO 19 ทำหน้าที่รับข้อมูลจากโมดูล MFRC522 เข้าสู่ ESP32
    - MOSI (Master Out Slave In): กำหนดใช้ GPIO 23 ทำหน้าที่ส่งข้อมูลจาก ESP32 ไปยังโมดูล MFRC522

- พินควบคุมการทำงาน (SDA & RST):
    - SDA / SS (Slave Select): กำหนดใช้ GPIO 5 ทำหน้าที่ส่งสัญญาณเลือกอุปกรณ์ในการสื่อสาร (Active Low)
    - RST (Reset): กำหนดใช้ GPIO 22 ทำหน้าที่ควบคุมการสั่งรีเซ็ตฮาร์ดแวร์เพื่อล้างสภาวะค้าง (State Machine Lockup) ของชิป MFRC522

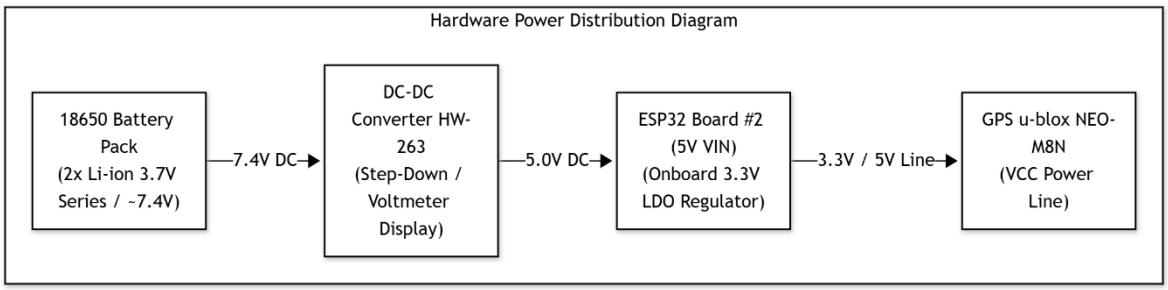
2. การจัดสรรพินสำหรับ ESP32 ตัวที่ 2 (Mobile GPS Tracker Node)

ไมโครคอนโทรลเลอร์ ESP32 ตัวที่ 2 ทำหน้าที่เป็นชุดติดตามเคลื่อนที่ติดยานพาหนะ เชื่อมต่อกับโมดูล GPS u-blox NEO-M8N ผ่านพอร์ตสื่อสารอนุกรมระดับฮาร์ดแวร์ HardwareSerial2 (UART2) โดยกำหนดใช้คำสั่ง HardwareSerial gpsSerial(2); การเลือกใช้ HardwareSerial แทนวงจรจำลองพอร์ต (SoftwareSerial) ช่วยลดภาระการทำงานของหน่วยประมวลผล (CPU Overhead) และป้องกันการชนกับพอร์ต UART0 ที่ใช้ส่งข้อมูล Debug ออกทาง Serial Monitor โดยจัดสรรพินดังนี้:

- GPS RX Pin (ฝั่ง ESP32): กำหนดใช้ GPIO 32 (ทำหน้าที่เป็น RX2) เชื่อมต่อเข้ากับพิน TX ของโมดูล GPS เพื่อรับสตรีมประโยคข้อมูล NMEA 0183 (เช่น $GNRMC) เข้ามาถอดรหัส
- GPS TX Pin (ฝั่ง ESP32): กำหนดใช้ GPIO 33 (ทำหน้าที่เป็น TX2) เชื่อมต่อเข้ากับพิน RX ของโมดูล GPS เพื่อส่งคำสั่งตั้งค่าหรือสั่งการไปยังโมดูล GPS

2.3.2 การจัดการพลังงานและปัญหากระแสไฟตกขณะส่งสัญญาณ Wi-Fi

ในการใช้อุปกรณ์ประมวลผลปลายทาง (Edge Node) แบบพกพาสำหรับติดยานพาหนะ ความเสถียรของระบบจ่ายไฟถือเป็นหัวใจสำคัญอย่างยิ่ง เนื่องจากระบบต้องเลี้ยงพลังงานให้กับทั้งไมโครคอนโทรลเลอร์ ESP32, โมดูล GPS u-blox NEO-M8N



2.3.2 การจัดการพลังงานและปัญหากระแสไฟตกขณะส่งสัญญาณ Wi-Fi

1. สถาปัตยกรรมการจัดสรรพลังงานทางฮาร์ดแวร์ (Hardware Power Architecture)

ระบบได้ออกแบบชุดจ่ายไฟแบบพกพาออกเป็น 2 ชั้น (Two-Stage Power Supply Architecture) เพื่อรองรับการทำงานในสภาวะเคลื่อนที่ ดังนี้

- แหล่งพลังงานต้นทาง (Primary Power Source): ใช้แบตเตอรี่ลิเธียมไอออน (Li-ion 18650) ความจุสูง แรงดัน nominal 3.7V จำนวน 2 ก้อน เชื่อมต่อกันแบบอนุกรมผ่านกระบะถ่าน ให้แรงดันไฟฟ้ารวมเฉลี่ยประมาณ 7.4V – 8.4V ซึ่งเป็นแหล่งจ่ายพลังงานที่มีอัตราการจ่ายกระแส (Discharge Rate) สูงเพียงพอต่อการใช้งานต่อเนื่อง
- วงจรรักษาระดับแรงดันไฟฟ้า (DC-DC Voltage Regulator): เชื่อมต่อแรงดันไฟจากแบตเตอรี่เข้าสู่โมดูล HW-263 (DC-DC Buck Converter / Step-Down Module) ซึ่งทำหน้าที่ แปลงลดแรงดันไฟฟ้าจาก 7.4V ให้คงที่อยู่ที่ 5.0V เพื่อป้อนเข้าสู่พิน VIN หรือ 5V ของบอร์ด ESP32 โดยโมดูล HW-263 มีหน้าจอแสดงผลดิจิทัล (7-Segment Voltmeter Display) สำหรับแสดงระดับแรงดันไฟฟ้าแบบเรียลไทม์ ทำให้ผู้ใช้สามารถตรวจวัดระดับพลังงานแบตเตอรี่คงเหลือก่อนออกปฏิบัติงานได้อย่างแม่นยำ

2. วิเคราะห์ปัญหากระแสไฟตกชั่วขณะ (Transient Power Spikes & Voltage Dips)

แม้การเลือกใช้ระบบแบตเตอรี่ร่วมกับโมดูล Buck Converter HW-263 จะให้กำลังไฟฟ้าที่มั่นคง แต่ในการทำงานจริงของไมโครคอนโทรลเลอร์ ESP32 จะเกิดสภาวะการดึงกระแสไฟฟ้าชั่วขณะที่รุนแรง ดังนี้

1. สภาวะ Wi-Fi RF Transmission Burst: ขณะที่ ESP32 เชื่อมต่อเครือข่ายไร้สายและส่งแพ็กเกจข้อมูล MQTT Payload ไปยัง EMQX Broker ภาคส่งสัญญาณความถี่วิทยุ (RF Power Amplifier) จะดึงกระแสไฟฟ้าสูงขึ้นทันทีในเวลาอันสั้น (Peak Current Spikes) ถึง 240–500 mA
2. ผลกระทบต่อระบบ (System Impact): หากใช้ระบบจ่ายไฟที่มีอัตราการตอบสนองต่อกระแสช้าหรือใช้เรกูเลเตอร์แบบเชิงเส้น (Linear Regulator) ทั่วไป การดึงกระแสกระชากนี้จะทำให้เกิดปรากฏการณ์แรงดันไฟเลี้ยงตกลงชั่วขณะ (Transient Voltage Dip / Ripple) ซึ่งส่งผลกระทบต่อระบบ 2 ประการ คือ
    - เกิดปัญหาระบบตัดการทำงานด้วยตนเอง (Brown-out Reset: BOR) บน ESP32 ทำให้ระบบรีบูตและขาดการติดต่อ
    - ส่งผลให้โมดูล GPS u-blox NEO-M8N สูญเสียสภาวะการล็อกสัญญาณดาวเทียม (GPS Fix Loss) เนื่องจากแรงดันไฟเลี้ยงชิปเกิดความไม่เสถียร

3. สรุปแนวทางการแก้ไขและความเสถียรของระบบ

จากการออกแบบระบบให้แยกโหนดการทำงานเป็น 2 ส่วน (Dual-Node Architecture) ร่วมกับการใช้โมดูลแปลงแรงดันไฟฟ้ากำลังสูง ทำให้การจัดการพลังงานมีความลงตัวและมีเสถียรภาพสูงสุด:

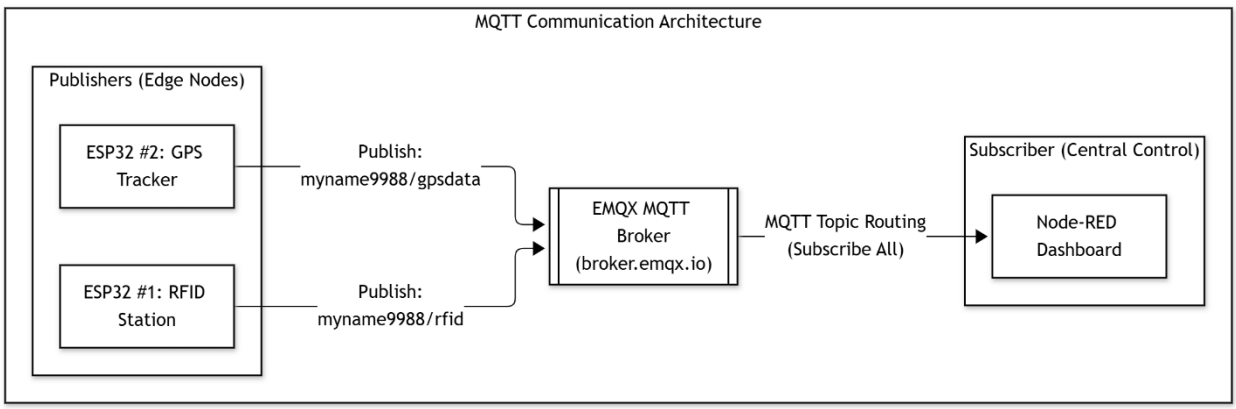
- โหนดสแกน RFID (Station Node): ติดตั้งประจำที่ ณ คลังสินค้า จ่ายไฟผ่านพอร์ต USB หรือ AC-DC Adapter มั่นคงสูง ทำให้หมดปัญหากระแสไฟตกรบกวนวงจรตรรกะของชิป MFRC522
- โหนดติดตามตำแหน่ง (Mobile GPS Tracking Node): การใช้โมดูล DC-DC Buck Converter (HW-263) ช่วยรักษาระดับแรงดันไฟเลี้ยง 5.0V ให้แก่ ESP32 และโมดูล GPS NEO-M8N ได้อย่างมั่นคงและทนต่อกระแสกระชากขณะส่งสัญญาณ Wi-Fi ได้ดีกว่าเรกูเลเตอร์ทั่วไป ทำให้โหนดติดตามสามารถอ่านพิกัดทางภูมิศาสตร์ คำนวณความเร็ว ระยะทาง และส่งข้อมูล MQTT ได้อย่างต่อเนื่องและเรียลไทม์ตลอดเส้นทางการขนส่ง

2.4 โปรโตคอลการสื่อสาร MQTT และระบบ EMQX Broker

โปรโตคอล MQTT (Message Queuing Telemetry Transport) เป็นโปรโตคอลการสื่อสารข้อมูลน้ำหนักเบา (Lightweight Messaging Protocol) ที่ถูกออกแบบบนสถาปัตยกรรม TCP/IP เหมาะสำหรับการแลกเปลี่ยนข้อมูลระหว่างอุปกรณ์ที่มีทรัพยากรจำกัด (Resource-constrained Devices) เช่น ไมโครคอนโทรลเลอร์ ESP32 และเครือข่ายที่มีแบนด์วิดท์ต่ำหรือมีความไม่แน่นอนสูง มีอัตราการใช้พลังงานและส่วนหัวของแพ็กเกจ (Header Overhead) ต่ำมากเพียง 2 ไบต์ จึงนิยมใช้เป็นมาตรฐานหลักในระบบอินเทอร์เน็ตของสรรพสิ่ง (IoT)

2.4.1 สถาปัตยกรรมแบบ Publisher / Broker / Subscriber

สถาปัตยกรรมของโปรโตคอล MQTT ไม่ได้ใช้การเชื่อมต่อแบบดึงข้อมูลตรงระหว่างผู้ส่งและผู้รับ (Client-Server) แต่ใช้โมเดลแบบ Publish / Subscribe โดยมีตัวกลางเรียกว่า Broker ทำหน้าที่บริหารจัดการเส้นทางและกระจายข้อความ



2.4.1 สถาปัตยกรรมแบบ Publisher / Broker / Subscriber

1. องค์ประกอบหลักในสถาปัตยกรรมของโครงงาน:
    - Publisher (ผู้ส่งข้อมูล): คือ ไมโครคอนโทรลเลอร์ ESP32 ทั้ง 2 ชุด ได้แก่
        - ESP32 #1 (RFID Station Node): ส่งสถานะการลงทะเบียนและการเปลี่ยนสภาวะการจัดส่ง (Start/Stop Tracking)
        - ESP32 #2 (Mobile GPS Tracker Node): ส่งข้อมูลพิกัด GPS, ความเร็ว และระยะทางสะสม
    - Broker (ตัวกลางจัดการข้อความ): เลือกใช้ EMQX Broker (broker.emqx.io) ซึ่งเป็น Enterprise-grade MQTT Broker ประสิทธิภาพสูง รองรับการเชื่อมต่อที่พร้อมกันเป็นจำนวนมาก (High Concurrency) และมีความหน่วงเวลา (Latency) ต่ำ
    - Subscriber (ผู้รับข้อมูล): คือ แพลตฟอร์ม Node-RED Dashboard ที่ทำการคอยดักฟัง (Subscribe) ข้อมูลจาก Topic ที่กำหนด เพื่อนำชุดข้อมูลไปประมวลผลและแสดงผลบนอินเทอร์เฟซผู้ใช้
2. ระบบหัวข้อข้อความ (MQTT Topic Hierarchy):

การส่งข้อความในโปรโตคอล MQTT จะต้องระบุ Topic เพื่อจำแนกหมวดหมู่ของข้อมูล โดยโครงงานนี้แบ่ง Topic ออกเป็น 2 ส่วนหลักเพื่อแยกประเภทข้อมูลจาก 2 โหนดอย่างเป็นระบบ:

- myname9988/rfid: สำหรับรับ-ส่งข้อมูลรหัสแท็ก UID และสถานะสวิตช์ติดตามจากโหนด RFID ประจำที่
- myname9988/gpsdata: สำหรับรับ-ส่งข้อมูลพิกัดภูมิศาสตร์และค่าทางกลศาสตร์จากโหนดติดตามเคลื่อนที่

2.4.2 โครงสร้างแพ็กเกจข้อมูลในรูปแบบ

การจัดส่งข้อมูลเซนเซอร์หลายค่าผ่านโปรโตคอล MQTT จำเป็นต้องจัดระเบียบข้อมูลให้อยู่ในรูปแบบโครงสร้างมาตรฐานที่ระบบปลายทางสามารถนำไปแยกแยะ (Parse) ได้ง่าย โครงงานนี้เลือกใช้รูปแบบ JSON (JavaScript Object Notation) ซึ่งเป็นรูปแบบข้อความเชิงโครงสร้างแบบ Key-Value Pair ที่มนุษย์อ่านเข้าใจได้ง่ายและใช้พื้นที่หน่วยความจำต่ำ

1. โครงสร้าง JSON Payload จากโหนดติดตามเคลื่อนที่ (ESP32 #2 - GPS Node):

ส่งผ่าน Topic myname9988/gpsdata ทุกๆ 2 วินาที เมื่ออยู่ในสภาวะกำลังจัดส่ง:

```
{
  "lat": 13.724561,
  "lng": 100.533202,
  "speed": 42.5,
  "dist": 12.84,
  "rssi": -65,
  "status": "TRACKING"
}
```

2. โครงสร้าง JSON Payload จากโหนดลงทะเบียนประจำที่ (ESP32 #1 - RFID Node):

ส่งผ่าน Topic myname9988/rfid เมื่อมีการทาบบัตรลงทะเบียน หรือสลับสถานะ:

```
{
  "uid": "A1B2C3D4",
  "action": "TOGGLE_STATUS",
  "status": "START"
}
```

3. คำอธิบายพารามิเตอร์ในชุดข้อมูล JSON:
    - uid (String): รหัสประจำตัวเฉพาะของแท็ก RFID ที่สแกนผ่าน MFRC522 (ขนาด 4 ไบต์)
    - lat (Double): พิกัดละติจูดปัจจุบันจากโมดูล u-blox NEO-M8N ในรูปแบบทศนิยม (Decimal Degrees)
    - lng (Double): พิกัดลองจิจูดปัจจุบันจากโมดูล u-blox NEO-M8N ในรูปแบบทศนิยม (Decimal Degrees)
    - speed (Float): ความเร็วในการเคลื่อนที่ของยานพาหนะขณะนั้น มีหน่วยเป็นกิโลเมตรต่อชั่วโมง (km/h)
    - dist (Float): ระยะทางสะสมรวมตลอดการขนส่ง คำนวณจากสูตร Haversine มีหน่วยเป็นกิโลเมตร (km)
    - rssi (Integer): ค่าความเข้มสัญญาณ Wi-Fi ที่ ESP32 เชื่อมต่อขณะนั้น มีหน่วยเป็น dBm (ใช้ประเมินคุณภาพการเชื่อมต่อเครือข่าย)
    - status (String): สถานะการทำงานปัจจุบันของระบบ เช่น START, STOP, TRACKING หรือ IDLE

2.5 แพลตฟอร์ม Node-RED และ Node-RED Dashboard

Node-RED เป็นแพลตฟอร์มซอฟต์แวร์แบบโอเพ่นซอร์ส (Open-source) ที่พัฒนาโดย IBM บนสภาพแวดล้อม Node.js ถูกออกแบบมาเพื่อการเชื่อมต่ออุปกรณ์ฮาร์ดแวร์ อินเทอร์เฟซโปรแกรมประยุกต์ (API) และบริการออนไลน์เข้าด้วยกันในรูปแบบการโปรแกรมเชิงภาพ (Visual Programming) Node-RED มีจุดเด่นคือการประมวลผลแบบตอบสนองต่อเหตุการณ์ (Event-driven Architecture) มีอัตราการกินทรัพยากรระบบต่ำ และรองรับโปรโตคอลมาตรฐาน IoT โดยเฉพาะ MQTT ได้อย่างสมบูรณ์แบบ จึงเหมาะแก่การนำมาใช้เป็นส่วนควบคุมกลางและระบบแสดงผล (Central Monitoring Server) ในโครงงานนี้

2.5.1 การทำงานเชิงโหนด

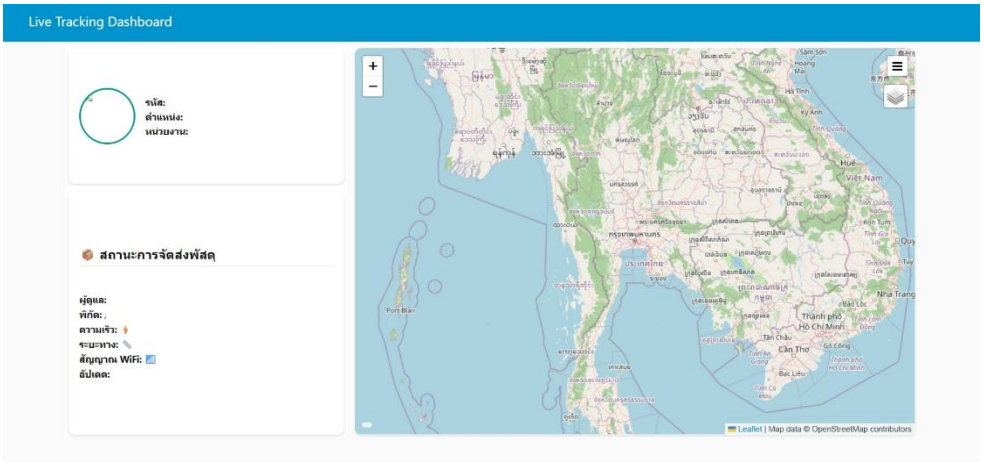
สถาปัตยกรรมซอฟต์แวร์ของ Node-RED อาศัยหลักการโปรแกรมเชิงการไหลของข้อมูล (Flow-based Programming: FBP) ซึ่งมองระบบเป็นเครือข่ายของ "โหนด" (Nodes) ที่เชื่อมต่อกันด้วย "สายสัญญาณ" (Wires) โดยข้อมูลจะถูกส่งผ่านระหว่างโหนดในรูปของวัตถุข้อความ (Message Object: msg) ซึ่งมีโครงสร้างหลักคือ

ลักษณะของโหนดการทำงานหลักที่ใช้ในโครงงาน:

- MQTT In Node (โหนดรับข้อมูล): ทำหน้าที่เป็น Subscriber เชื่อมต่อกับ EMQX Broker (broker.emqx.io) ผ่านพอร์ต 1883 เพื่อคอยดักรับข้อมูลจาก Topic myname9988/rfid (จากโหนด RFID ประจำที่) และ Topic myname9988/gpsdata (จากโหนด GPS ติดรถ)
- JSON Node (โหนดแปลงโครงสร้างข้อมูล): ทำหน้าที่แปลงชุดข้อความ JSON String ที่รับมาจากเครือข่าย ให้กลายเป็น JavaScript Object เพื่อให้สามารถเข้าถึงตัวแปรภายในผ่านคำสั่ง msg.payload.lat หรือ msg.payload.uid ได้โดยตรง
- Function Node (โหนดประมวลผลตรรกะ): ใช้เขียนโค้ดภาษา JavaScript เพื่อประมวลผลเงื่อนไข คัดแยกข้อมูล ตรวจสอบสภาวะการจัดส่ง (status == 'START') และการจัดฟอร์แมตข้อมูลก่อนส่งต่อไปยังส่วนแสดงผล
- UI Nodes (node-red-dashboard): โหนดคอมโพเนนต์สำหรับสร้างหน้าต่างโต้ตอบกับผู้ใช้ (User Interface) เช่น ข้อความ (Text), เกจวัด (Gauge), การ์ด (Card) และแผนที่ (Worldmap)

2.5.2 การจัดการแผนที่โต้ตอบและการ์ดแสดงผล

ส่วนแสดงผลผู้ใช้งาน (Dashboard) พัฒนาขึ้นโดยใช้พาเลตเสริม node-red-dashboard ร่วมกับ node-red-contrib-web-worldmap เพื่อสร้างหน้าต่างแสดงผลแบบเว็บแอปพลิเคชันที่รองรับการใช้งานทั้งบนคอมพิวเตอร์และอุปกรณ์เคลื่อนที่ โดยแบ่งการแสดงผลข้อมูลออกเป็น 3 ส่วนหลัก ดังนี้:



2.5.2 การจัดการแผนที่โต้ตอบและการ์ดแสดงผล

1. แผนที่โต้ตอบ (Interactive Map Display):
    - ใช้โหนด Worldmap ร่วมกับฐานข้อมูลแผนที่ OpenStreetMap (OSM) เพื่อแสดงตำแหน่งของพัสดุและยานพาหนะขนส่งบนแผนที่แบบเรียลไทม์
    - รับค่าพิกัด lat และ lng จาก Topic myname9988/gpsdata เพื่อเลื่อนหมุด (Marker) ไปยังตำแหน่งล่าสุดโดยอัตโนมัติ
    - มีระบบสร้างเส้นทางการเดินทาง (Polyline Trail) ตามพิกัดย้อนหลัง เพื่อให้ผู้ดูแลระบบสามารถตรวจสอบเส้นทางขนส่งจริงได้
2. การ์ดข้อมูลผู้จัดส่งและพัสดุ (Staff / Item Information Card):
    - รับข้อมูลรหัส UID จาก Topic myname9988/rfid เมื่อมีการสแกนแท็ก ณ โหนดประจำคลังสินค้า (Station Node)
    - แสดงผลรหัสบัตร UID, ชื่อ-นามสกุล ของพนักงานผู้รับผิดชอบรอบการขนส่ง และแสดงสถานะเปิด/ปิดรอบการติดตาม (START / STOP) ในรูปแบบป้ายไฟสี (Status Badge)
3. การ์ดสถานะการจัดส่งพัสดุแบบเรียลไทม์ (Delivery Status Card):
    - แสดงพารามิเตอร์ทางกลศาสตร์และการสื่อสารที่ส่งมาจากโหนดติดตามเคลื่อนที่ (Mobile GPS Node)
    - แสดงค่าพิกัดละติจูด-ลองจิจูด, ความเร็วเคลื่อนที่ปัจจุบัน (หน่วย km/h), ระยะทางสะสมรวมตลอดเส้นทาง (หน่วย km) และค่าความเข้มสัญญาณ Wi-Fi (rssi หน่วย dBm)
    - มีระบบประทับเวลา (Timestamp) ของข้อมูลชุดล่าสุด เพื่อยืนยันว่าระบบกำลังทำการติดตามอยู่จริงในรูปแบบเรียลไทม์

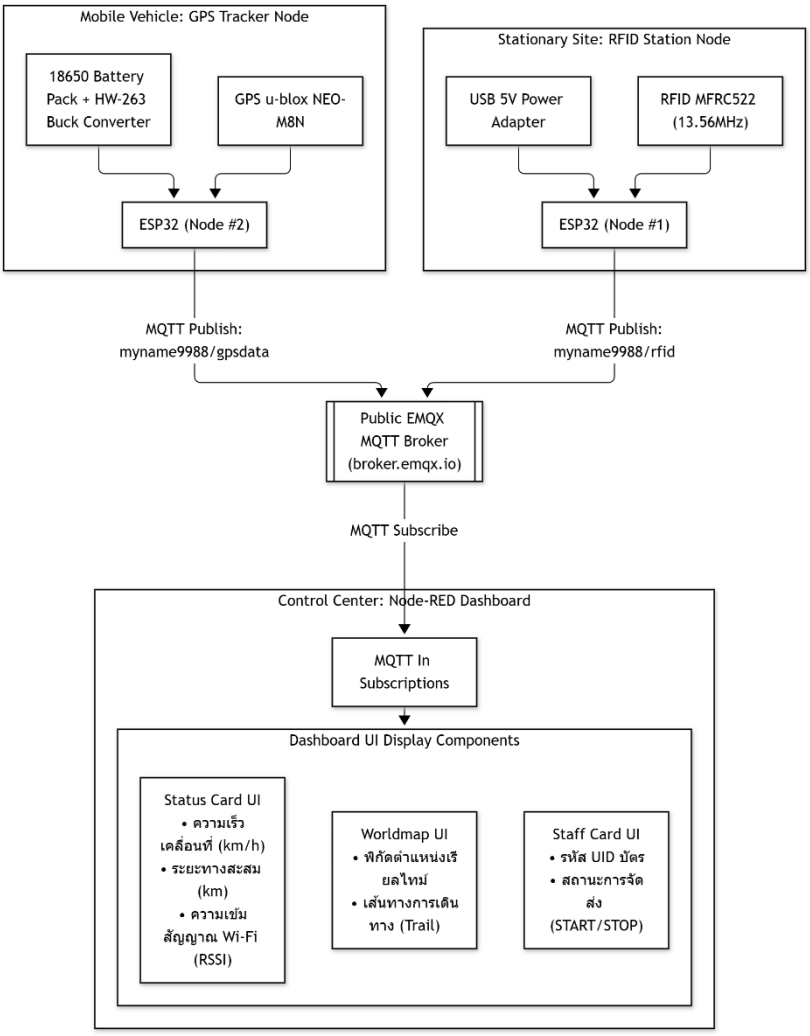
บทที่ 3

การออกแบบและพัฒนาระบบ

การพัฒนาระบบติดตามสินค้าด้วย RFID และ GPS แบบเรียลไทม์ (Real-Time Product Tracking System) ในโครงงานนี้ ได้ออกแบบสถาปัตยกรรมระบบฮาร์ดแวร์แบบ 2 โหนดอิสระ (Dual-Node Architecture) เพื่อแยกหน้าที่การประมวลผลระหว่างการระบุตัวตน ณ คลังสินค้ากับการติดตามตำแหน่งยานพาหนะขณะเดินทาง ซึ่งช่วยตัดปัญหาสัญญาณรบกวนทางไฟฟ้าระหว่างโมดูล เพิ่มเสถียรภาพการทำงาน และรองรับการนำไปติดตั้งใช้งานจริงในงานโลจิสติกส์ได้อย่างมีประสิทธิภาพ

3.1 ภาพรวมสถาปัตยกรรมของระบบ

ระบบประกอบด้วย 3 ส่วนหลัก ได้แก่ โหนดฮาร์ดแวร์ประมวลผลปลายทาง (Edge Nodes), ตัวกลางรับ-ส่งข้อความ (MQTT Broker) และแพลตฟอร์มแสดงผลศูนย์ควบคุม (Node-RED Dashboard)



3.1 ภาพรวมสถาปัตยกรรมของระบบ

1. โหนดตรวจสอบประจำที่ (RFID Station Node - ESP32 #1): ติดตั้งไว้ ณ จุดรับ-ส่งสินค้า หรือคลังสินค้า ทำหน้าที่อ่านรหัสแท็ก RFID (UID) จากพนักงานหรือพัสดุผ่านโมดูล MFRC522 เมื่อสแกนสำเร็จจะส่งข้อมูลโครงสร้าง JSON ไปยัง MQTT Broker ใน Topic myname9988/rfid เพื่อสั่งการเปลี่ยนสถานะการติดตาม (START หรือ STOP)
2. โหนดติดตามตำแหน่งเคลื่อนที่ (Mobile GPS Tracking Node - ESP32 #2): ติดตั้งไปกับยานพาหนะขนส่ง ทำหน้าที่ดึงพิกัดทางภูมิศาสตร์จากโมดูล u-blox NEO-M8N คำนวณความเร็วและระยะทางสะสม แล้วแพ็กเป็น JSON Payload ส่งไปยัง MQTT Broker ใน Topic myname9988/gpsdata ทุกๆ 2 วินาที
3. ระบบรับ-ส่งข้อมูลและแสดงผล (EMQX Broker & Node-RED): EMQX Broker ทำหน้าที่รับและกระจายข้อความไปยัง Node-RED Dashboard ซึ่งทำหน้าที่คอยดักฟัง (Subscribe) ข้อมูลจากทั้งสอง Topic เพื่อรวมข้อมูล (Data Fusion) และแสดงผลบนแผนที่โต้ตอบและการ์ดสรุปสถานะแบบเรียลไทม์

3.2 การออกแบบส่วนฮาร์ดแวร์

การออกแบบฮาร์ดแวร์ของระบบติดตามสินค้าด้วย RFID และ GPS แบบเรียลไทม์ ใช้สถาปัตยกรรมแบบ 2 โหนดอิสระ (Dual-Node Architecture) เพื่อแยกแยะหน้าที่การทำงาน (Task Separation) ออกเป็น 2 ส่วนทางกายภาพอย่างชัดเจน ได้แก่ สถานีลงทะเบียนประจำที่ (RFID Station Node) และ ชุดติดตามตำแหน่งพกพาติดยานพาหนะ (Mobile GPS Tracker Node) การแยกโครงสร้างฮาร์ดแวร์ช่วยลดความซับซ้อนของการต่อสายสัญญาณ เพิ่มเสถียรภาพในการจ่ายไฟ และทำให้การติดตั้งใช้งานในเชิงปฏิบัติมีความยืดหยุ่นสูง

รายการอุปกรณ์ / โมดูล	รุ่น / สเปกฮาร์ดแวร์	โหนดที่สังกัด	หน้าที่การทำงานหลักในระบบ
ไมโครคอนโทรลเลอร์ #1	ESP32-WROOM-32	โหนดที่ 1 (Station)	ประมวลผลข้อมูลสแกนบัตร RFID แส่งข้อมูลสลงทะเบียนเข้าสู่ระบบ
โมดูลอ่านบัตร RFID	MFRC522 (13.56 MHz, SPI)	โหนดที่ 1 (Station)	ตรวจจับแท็ก RFID อ่านค่า UID เพื่อระบุรหัสพนักงานหรือพัสดุ
แหล่งจ่ายไฟโหนดที่ 1	USB Adapter 5V DC	โหนดที่ 1 (Station)	จ่ายกระแสไฟฟ้าห้ไมโครคอนโทรลเลอร์ ESP32 #1
ไมโครคอนโทรลเลอร์ #2	ESP32-WROOM-32	โหนดที่ 2 (Mobile)	อ่านค่าพิกัด GPS คำนวณระยะทางสะสม ความเร็ว และส่งข้อมูลผ่าน Wi-Fi/MQTT
โมดูลรับพิกัดดาวเทียม	u-blox NEO-M8N (UART)	โหนดที่ 2 (Mobile)	สื่อสารกับดาวเทียม GPS เพื่อรับสตรีมประโยคข้อมูล NMEA 0183
แบตเตอรี่พลังงาน	Li-ion 18650 (2 ก้อนอนุกรม ~7.4V)	โหนดที่ 2 (Mobile)	เป็นแหล่งพลังงานพกพาสำหรับชุดติดตาม
โมดูลแปลงแรงดัน	Buck Converter HW-263	โหนดที่ 2 (Mobile)	แปลงแรงดันไฟจากแบตเตอรี่ 7.4V ลงเหลือ 5.0V DC จ่ายเข้าพิน

3.2 การออกแบบส่วนฮาร์ดแวร์

3.2.1 การต่อวงจรและการจัดสรรพินสัญญาณ

การออกแบบวงจรอิเล็กทรอนิกส์ของระบบติดตามสินค้าแบบเรียลไทม์ ใช้สถาปัตยกรรมแบบ 2 โหนดอิสระ (Dual-Node Architecture) โดยแบ่งการทำงานออกเป็น 2 ส่วนแยกจากกันทางกายภาพ เพื่อความคล่องตัวในการติดตั้งใช้งานจริง ณ คลังสินค้าและการติดตามยานพาหนะระหว่างการจัดส่ง ดังมีรายละเอียดการต่อวงจรของแต่ละโหนดดังนี้

1. การต่อวงจรโหนดที่ 1: RFID Station Node (ชุดลงทะเบียนประจำที่)

โหนดที่ 1 ทำหน้าที่เป็นสถานีรับลงทะเบียนสินค้า/พนักงาน ติดตั้งประจำที่ ณ จุดมอนิเตอร์หรือคลังสินค้า ใช้พลังงานจากแหล่งจ่ายไฟภายนอกผ่านอะแดปเตอร์ USB 5V ประมวลผลด้วยไมโครคอนโทรลเลอร์ ESP32 (บอร์ดที่ 1) เชื่อมต่อกับโมดูลอ่านแท็ก RFID MFRC522 ผ่านบัสการสื่อสารความเร็วสูง

VSPI (Serial Peripheral Interface)

การจัดสรรพินสัญญาณและการเชื่อมต่อวงจร (ESP32 #1 + MFRC522):

- VCC (3.3V): ต่อเข้ากับพินจ่ายแรงดัน 3.3V ของ ESP32 #1 (ห้ามต่อไฟ 5V โดยตรงเพื่อป้องกันชิป MFRC522 เสียหาย)
- GND: ต่อเข้ากับพิน Ground (GND) ของ ESP32 #1
- SDA / SS: ต่อเข้ากับ GPIO 5 ทำหน้าที่ส่งสัญญาณเลือกอุปกรณ์ (Slave Select / Active Low)
- SCK: ต่อเข้ากับ GPIO 18 ทำหน้าที่รับสัญญาณนาฬิกาบัส VSPI
- MISO: ต่อเข้ากับ GPIO 19 ทำหน้าที่เป็นสายส่งข้อมูล Master In Slave Out เข้าสู่ ESP32 #1
- MOSI: ต่อเข้ากับ GPIO 23 ทำหน้าที่เป็นสายส่งข้อมูล Master Out Slave In ไปยัง MFRC522
- RST: ต่อเข้ากับ GPIO 22 ทำหน้าที่เป็นสายควบคุมการรีเซ็ตฮาร์ดแวร์ (Hardware Force Reset) เพื่อแก้ปัญหาชิปค้างเมื่อมีการดึงกระแสไฟช่วงส่งสัญญาณ Wi-Fi

พินโมดูล RFID MFRC522	พิน ESP32 (บอร์ดที่ 1)	ทิศทางสัญญาณ	หน้าที่การทำงาน
VCC	3.3V	Input (Power)	จ่ายไฟเลี้ยงโมดูล 3.3V
GND	GND	-	กราวด์ร่วมระบบ
SDA (SS)	GPIO 5	Output	สัญญาณเลือกอุปกรณ์
SCK	GPIO 18	Output	สัญญาณนาฬิกาบัส SPI
MISO	GPIO 19	Input	สายส่งข้อมูล
MOSI	GPIO 23	Output	สายส่งข้อมูล
RST	GPIO 22	Output	สายสั่งการรีเซ็ต ฮาร์ดแวร์

ตารางที่ 3.1 สรุปการเชื่อมต่อพินสำหรับโหนดที่ 1

2. การต่อวงจรโหนดที่ 2: Mobile GPS Tracker Node (ชุดพกพาติดยานพาหนะ)

โหนดที่ 2 ทำหน้าที่เป็นอุปกรณ์ติดตามตำแหน่งพิกัดภูมิศาสตร์เคลื่อนที่ ติดตั้งไว้บนยานพาหนะจัดส่งพัสดุ ประมวลผลด้วยไมโครคอนโทรลเลอร์ ESP32 (บอร์ดที่ 2) เชื่อมต่อกับโมดูลรับพิกัดดาวเทียม u-blox NEO-M8N ผ่านพอร์ตสื่อสารอนุกรมฮาร์ดแวร์ HardwareSerial2 (UART2) เพื่ออ่านประโยคข้อมูล NMEA เข้ามาประมวลผล

ระบบจัดการพลังงานและการเชื่อมต่อวงจร (Power Distribution): เนื่องจากเป็นชุดเคลื่อนที่ จึงใช้พลังงานจากแบตเตอรี่ชนิด Li-ion 18650 จำนวน 2 ก้อน อนุกรมกัน (แรงดันไฟฟ้ารวมประมาณ 7.4V - 8.4V DC) ต่อเข้ากับโมดูลปรับลดแรงดันสวิตชิ่ง DC-DC Buck Converter HW-263 เพื่อแปลงแรงดันให้คงที่ที่ 5.0V DC ก่อนจ่ายเข้าพิน VIN ของ ESP32 #2

- วงจรภาคจ่ายไฟ: ขั้วจ่ายไฟเอาต์พุตบวก (OUT+) ของโมดูล HW-263 ต่อเข้าพิน VIN ของ ESP32 โหนดที่ 2 และขั้วจ่ายไฟเอาต์พุตลบ (OUT-) ต่อเข้าพิน GND ของ ESP32 โหนดที่ 2VCC (GPS): ต่อเข้ากับพิน 3.3V (หรือ 5V) ของ ESP32 #2
- GND (GPS): ต่อเข้ากับพิน Ground (GND) ร่วมของ ESP32 #2
- TX (GPS): ต่อเข้ากับ GPIO 32 (UART2 RX2) ของ ESP32 #2 เพื่อรับสตรีมข้อมูล NMEA
- RX (GPS): ต่อเข้ากับ GPIO 33 (UART2 TX2) ของ ESP32 #2 เพื่อส่งคำสั่งตั้งค่าไปยังโมดูล GPS

อุปกรณ์ / โมดูล	พินของอุปกรณ์	พิน ESP32 (บอร์ดที่ 2)	หน้าที่และคำอธิบายการทำงาน
แบตเตอรี่ 18650 (7.4V)	ขั้วบวก / ขั้วลบ	IN+ / IN- (โมดูล HW-263)	ป้อนแรงดันไฟฟ้าระดับ 7.4V เข้าสู่ Buck Converter
โมดูล HW-263 (Buck)	OUT+ / OUT-	VIN / GND (ESP32 #2)	แปลงไฟเป็น 5.0V DC จ่ายเข้าไมโครคอนโทรลเลอร์
GPS NEO-M8N	VCC	3.3V (หรือ 5V)	จ่ายไฟเลี้ยงโมดูล GPS
	GND	GND	สายกราวด์ร่วมของโหนดที่ 2
	TX	GPIO 32 (UART2 RX2)	รับสตรีมประโยคข้อมูล NMEA เข้าสู่ ESP32 #2
	RX	GPIO 33 (UART2 TX2)	ส่งคำสั่งตั้งค่าพารามิเตอร์ไปยังโมดูล GPS

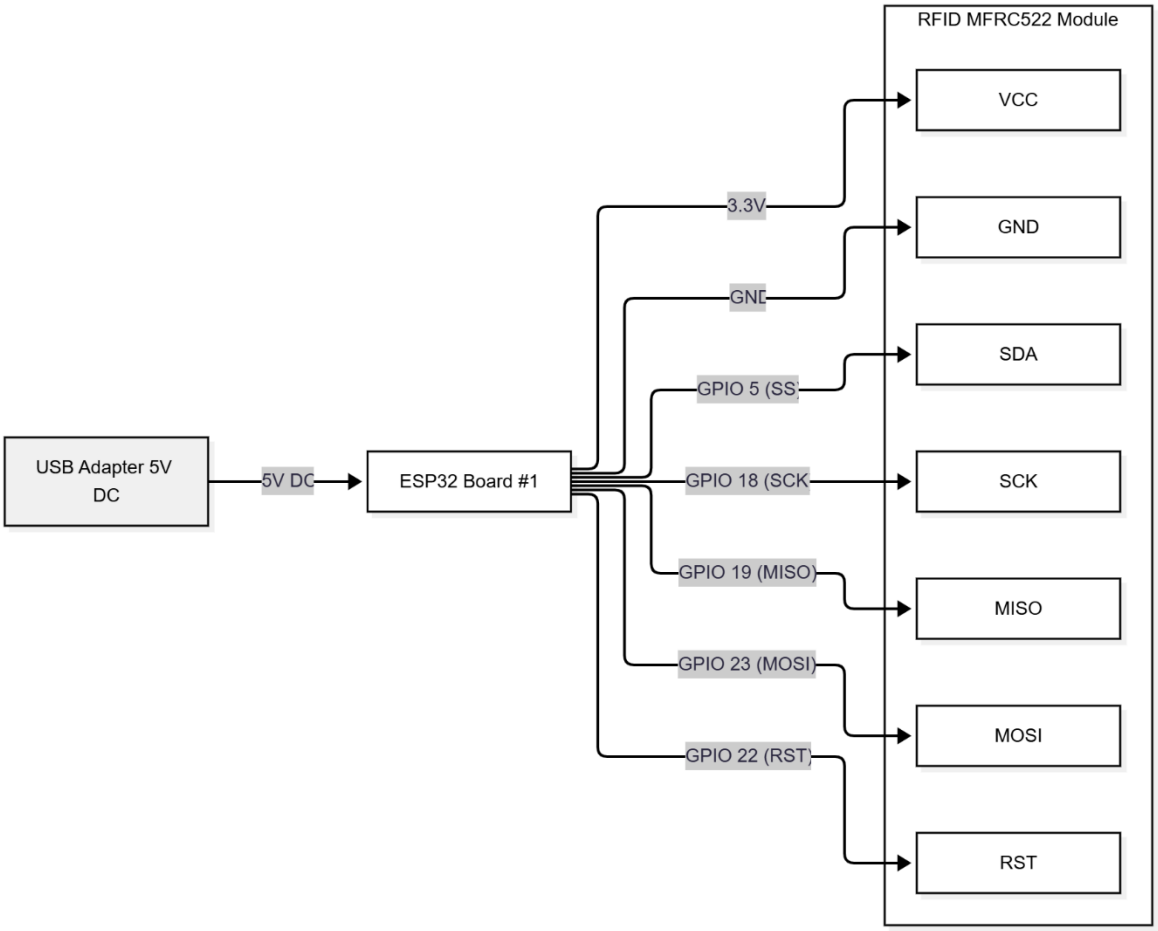
ตารางที่ 3.2 สรุปการเชื่อมต่อพินสำหรับโหนดที่ 2

3.2.2 ผังการเชื่อมต่อสายสัญญาณและระบบจ่ายไฟ

การออกแบบผังวงจรไฟฟ้าและการจ่ายพลังงาน (Wiring & Power Schematic Diagram) ของระบบติดตามสินค้าแบบเรียลไทม์ ใช้หลักการแยกวงจรไฟเลี้ยงและการประมวลผลออกเป็น 2 โหนดอิสระอย่างเด็ดขาด เพื่อป้องกันปัญหาสัญญาณรบกวนข้ามระบบ (Cross-talk Noise) และปัญหากระแสไฟฟ้าไม่พอเพียงขณะส่งข้อมูลผ่านสัญญาณ Wi-Fi โดยรายละเอียดการต่อวงจรและโครงสร้างทางไฟฟ้าของแต่ละโหนดมีดังนี้

1. ผังการเชื่อมต่อโหนดที่ 1: RFID Station Node

หนดที่ 1 ได้รับการออกแบบให้รับพลังงานไฟฟ้าต่อเนื่องจากอะแดปเตอร์ USB 5V DC เข้าสู่ไมโครคอนโทรลเลอร์ ESP32 บอร์ดที่ 1 จากนั้นชุดปรับแรงดันบนบอร์ด (On-board LDO Regulator) จะทำการแปลงแรงดันลงเหลือ 3.3V DC เพื่อจ่ายไฟเลี้ยงให้แก่ชิปประมวลผลหลักและโมดูล RFID MFRC522



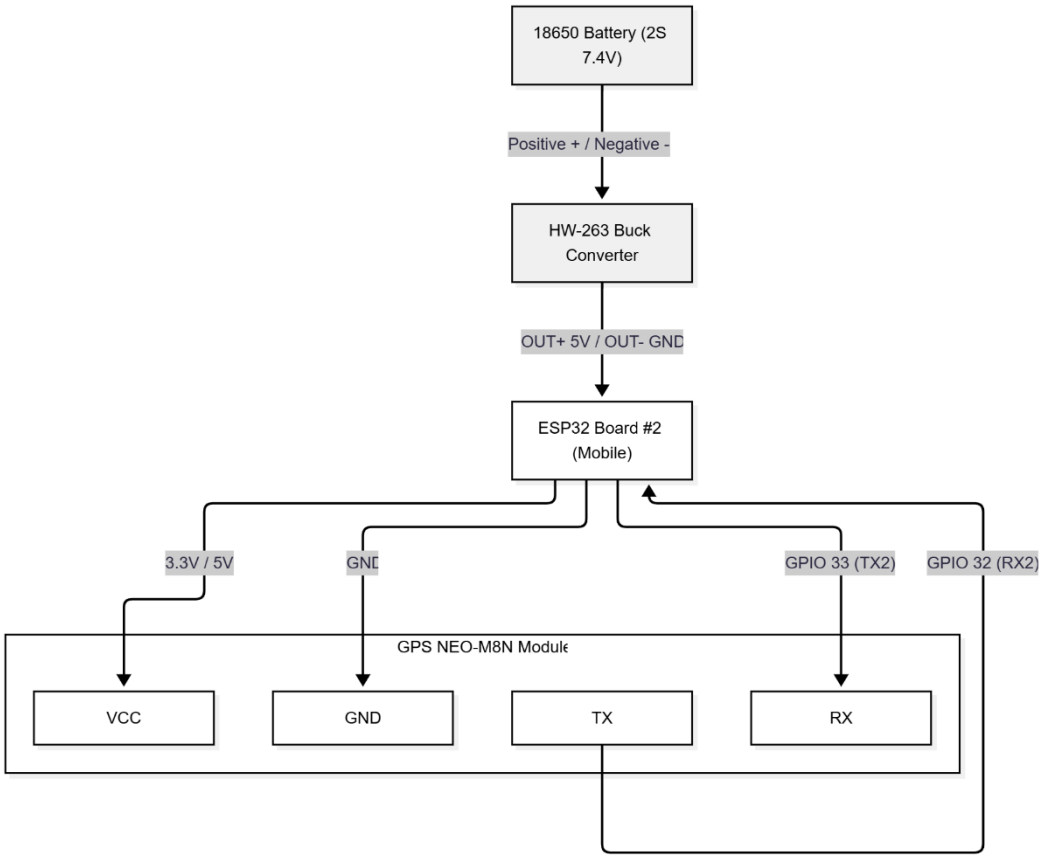
1. ผังการเชื่อมต่อโหนดที่ 1: RFID Station Node

รายละเอียดการเดินสายสัญญาณโหนดที่ 1:

- สายไฟเลี้ยง (Power Lines):
    - พิน 3.3V ของ ESP32 #1 → พิน VCC ของ MFRC522 (ห้ามต่อไฟ 5V โดยเด็ดขาด)
    - พิน GND ของ ESP32 #1 → พิน GND ของ MFRC522
- สายบัสสื่อสาร VSPI (Signal Lines):
    - พิน GPIO 5 (SS/SDA) → พิน SDA ของ MFRC522 (Active Low Line)
    - พิน GPIO 18 (SCK) → พิน SCK ของ MFRC522 (Clock Line)
    - พิน GPIO 19 (MISO) → พิน MISO ของ MFRC522 (Data Line: Slave to Master)
    - พิน GPIO 23 (MOSI) → พิน MOSI ของ MFRC522 (Data Line: Master to Slave)
    - พิน GPIO 22 (RST) → พิน RST ของ MFRC522
      (Hardware Control Line สั่งล้างสภาวะค้าง)

2. ผังการเชื่อมต่อโหนดที่ 2: Mobile GPS Tracker Node (ชุดพกพาติดยานพาหนะ)

โหนดที่ 2 ออกแบบมาสำหรับการใช้งานภาคสนามบนยานพาหนะ ใช้พลังงานจากแบตเตอรี่ชนิด Li-ion 18650 จำนวน 2 ก้อนต่ออนุกรมกัน ให้แรงดันไฟฟ้ารวมประมาณ 7.4 V (สูงสุด 8.4 V เมื่อชาร์จเต็ม) ต่อเข้าสู่โมดูล DC-DC Buck Converter HW-263 เพื่อแปลงแรงดันให้คงที่อยู่ที่ 5 V ก่อนจ่ายเข้าพิน VIN ของ ESP32 บอร์ดที่ 2



2. ผังการเชื่อมต่อโหนดที่ 2: Mobile GPS Tracker Node

รายละเอียดการเดินสายสัญญาณโหนดที่ 2:

- วงจรการจัดการพลังงาน (Power Management Wiring):
  - ขั้วบวก (+) ของชุดแบตเตอรี่ 18650 → พิน IN+ ของโมดูล HW-263
  - ขั้วลบ (-) ของชุดแบตเตอรี่ 18650 → พิน IN- ของโมดูล HW-263
  - พิน OUT+ (ปรับไว้ที่ 5.0V DC) ของ HW-263 → พิน VIN (5V Input) ของ ESP32 #2
  - พิน OUT- ของ HW-263 → พิน GND ของ ESP32 #2
- วงจรเชื่อมต่อโมดูล GPS (HardwareSerial2 Wiring):
  - พิน 3.3V (หรือ 5V) ของ ESP32 #2 → พิน VCC ของโมดูล GPS NEO-M8N
  - พิน GND ของ ESP32 #2 → พิน GND ของโมดูล GPS NEO-M8N
  - พิน TX ของโมดูล GPS → พิน GPIO 32 (UART2 RX2) ของ ESP32 #2 (รับข้อมูล NMEA)
  - พิน RX ของโมดูล GPS → พิน GPIO 33 (UART2 TX2) ของ ESP32 #2 (ส่งคำสั่งตั้งค่า)

3. มาตรการความเสถียรของระบบสายสัญญาณและวงจรจ่ายไฟ

1. การรวมจุดลงกราวด์ร่วม (Common Grounding Principle): อุปกรณ์ทุกชนิดภายในโหนดเดียวกันต้องมีการเชื่อมต่อพิน GND ถึงกันอย่างสมบูรณ์เพื่อสร้างระดับแรงดันอ้างอิง 0V เดียวกัน หากจุด GND ไม่ถึงกัน จะส่งผลให้การรับส่งข้อมูลผ่านบัส SPI หรือ UART เกิดความผิดพลาด (Data Corruption)
2. การควบคุมความยาวสายสัญญาณ: เนื่องจากบัส VSPI ของโมดูล MFRC522 ทำงานที่ความถี่สูง สายสัญญาณทั้ง 5 เส้น (SDA, SCK, MISO, MOSI, RST) จะต้องมีความยาวไม่เกิน 15-20 เซนติเมตร เพื่อลดค่าความจุไฟฟ้าแฝง (Parasitic Capacitance) และป้องกันสัญญาณรบกวนภายนอก
3. การป้องกันแรงดันไฟตกชั่วขณะ (Voltage Dip Protection): ในช่วงที่ ESP32 ทำการส่งข้อมูลผ่าน Wi-Fi เข้าสู่ MQTT Broker จะเกิดกระแสไฟกระชาก (Current Spike) การใช้โมดูล DC-DC Buck Converter HW-263 ร่วมกับแบตเตอรี่ 18650 ในโหนดที่ 2 ช่วยให้สามารถจ่ายกระแสต่อเนื่องได้สูงถึง 2A-3A ซึ่งเพียงพอและป้องกันอาการรีบูตตัวเองของ ESP32 ได้อย่างมีประสิทธิภาพ

3.3 การพัฒนาเฟิร์มแวร์บนไมโครคอนโทรลเลอร์

การพัฒนาเฟิร์มแวร์สำหรับไมโครคอนโทรลเลอร์ ESP32 ทั้งสองโหนด ดำเนินการบนโปรแกรม Arduino IDE ด้วยภาษา C/C++ โดยเน้นการออกแบบการทำงานแบบจัดสรรทรัพยากรฮาร์ดแวร์อย่างมีประสิทธิภาพ (Hardware Resource Management) และการประมวลผลแบบ Non-blocking I/O เพื่อให้ระบบสามารถอ่านค่าจากเซนเซอร์ ควบคุมสภาวะการทำงาน และรับ-ส่งข้อมูลผ่านเครือข่ายไร้สาย Wi-Fi และโปรโตคอล MQTT ได้อย่างต่อเนื่องเรียลไทม์

3.3.1 อัลกอริทึมการสแกนบัตรสลับสภาวะการทำงาน

ในส่วนการทำงานของโหนดรับลงทะเบียนและควบคุมการติดตาม (RFID Station) เฟิร์มแวร์ทำหน้าที่อ่านค่ารหัสประจำตัวบัตร (UID) จากโมดูล RFID MFRC522 เพื่อสลับสภาวะการทำงานของระบบระหว่างเริ่มติดตาม (Start Tracking) และหยุดติดตาม (Stop Tracking)

1. ขั้นตอนการอ่านค่า UID จากแท็ก RFID

1. เรียกใช้ฟังก์ชัน rfid.PICC_IsNewCardPresent() เพื่อตรวจสอบว่ามีแท็ก RFID เข้ามาในระยะตรวจจับของสายอากาศหรือไม่
2. เรียกใช้ฟังก์ชัน rfid.PICC_ReadCardSerial() เพื่ออ่านสตรีมไบต์รหัส UID ขนาด 4 ไบต์
3. แปลงค่าไบต์อาร์เรย์ (rfid.uid.uidByte[i]) ให้อยู่ในรูปสตริงรหัสฐานสิบหก (Hexadecimal String)
4. ปรับรูปแบบสตริงให้เป็นตัวพิมพ์ใหญ่ (Upper Case) ด้วยฟังก์ชัน toUpperCase() เพื่อใช้เป็นคีย์อ้างอิงประจำตัวพัสดุหรือเจ้าหน้าที่

2. อัลกอริทึมการสลับสภาวะ (State Toggle Logics)

ระบบใช้ตัวแปรประเภท Boolean isTracking ในการจัดสภาวะการทำงาน เมื่อตรวจพบการทาบบัตร ตัวแปรจะถูกสลับค่าผ่านคำสั่ง isTracking = !isTracking; โดยมีเงื่อนไขดังนี้:

- สภาวะเริ่มต้นติดตาม (isTracking == true):
  - บันทึกค่า UID เข้าสู่ตัวแปร activeUID
  - เคลียร์ค่าระยะทางสะสม totalDistance = 0.0 และรีเซ็ตพิกัดอ้างอิง previousLat = 0.0, previousLng = 0.0
  - ตั้งค่าเวลาเริ่มต้นส่งข้อมูล lastSendTime = millis() - sendInterval; เพื่อบังคับให้ระบบส่งข้อมูล MQTT แพ็กเกจแรกทันที
- สภาวะสิ้นสุดการติดตาม (isTracking == false):
  - สร้างโครงสร้างข้อมูล JSON สรุปการจัดส่ง เช่น {"uid":"A1B2C3D4", "dist":12.84, "rssi":-65, "status":"completed"}
  - ส่งแพ็กเกจ MQTT แจ้งเตือนไปยัง MQTT Broker เพื่ออัปเดตสถานะบนหน้าจอ Dashboard ว่าการจัดส่งเสร็จสิ้น

```
// ตัวอย่างโค้ดอัลกอริทึมสแกนบัตรสลับสภาวะ
if (rfid.PICC_IsNewCardPresent() && rfid.PICC_ReadCardSerial()) {
  String uidString = "";
  for (byte i = 0; i < rfid.uid.size; i++) {
    uidString += String(rfid.uid.uidByte[i] < 0x10 ? "0" : "");
    uidString += String(rfid.uid.uidByte[i], HEX);
  }
  uidString.toUpperCase();
  isTracking = !isTracking; // สลับสภาวะการทำงาน
  if (isTracking) {
    activeUID = uidString;
    totalDistance = 0.0;
    previousLat = 0.0;
    previousLng = 0.0;
    Serial.println("\n[▶] START TRACKING");
  } else {
```

```
    Serial.println("\n[■] STOP TRACKING");
    // สร้างและส่ง JSON Payload แจ้งเตือนสภาวะ completed
  }
  delay(2000); // หน่วงเวลาป้องกันการอ่านบัตรซ้ำซ้อน
  forceResetRFID();
}
```

3.3.2 อัลกอริทึมการคำนวณระยะทางสะสมและความเร็วจากพิกัด GPS

โหนดติดตามตำแหน่งเคลื่อนที่ (Mobile GPS Tracker Node) ใช้อนุกรมสื่อสาร HardwareSerial2 บนพิน GPIO 32 (RX2) และ GPIO 33 (TX2) ความเร็ว 9600 bps เพื่อรับสตรีมประโยคข้อมูล NMEA จากโมดูล GPS u-blox NEO-M8N

1. การถอดรหัสประโยค NMEA ด้วยไลบรารี TinyGPS++

ในลูปการทำงานหลัก (loop) เฟิร์มแวร์จะคอยตรวจสอบข้อมูลอนุกรมขาส่งจากโมดูล GPS และส่งเข้าสู่ฟังก์ชันถอดรหัสผ่านคำสั่ง gps.encode(gpsSerial.read()) เมื่อไลบรารีถอดรหัสประโยค $GPRMC$ หรือ $GPGGA$ สำเร็จ จะทำการอัปเดตสถานะ gps.location.isValid() เป็น true

2. การคำนวณความเร็วและระยะทางสะสม (Haversine Formula)

- ความเร็ว (Speed): อ่านค่าความเร็วเชิงดินระดับกิโลเมตรต่อชั่วโมงได้โดยตรงจากฟังก์ชัน gps.speed.kmph()
- ระยะทางสะสม (Cumulative Distance): คำนวณระยะทางระหว่างจุดพิกัดปัจจุบัน (Lat_2, Lng_2) กับพิกัดก่อนหน้า (Lat_1, Lng_1) โดยใช้สูตรฮาเวอร์ซีน (Haversine Formula) ซึ่งไลบรารี TinyGPS++ ให้บริการผ่านฟังก์ชัน TinyGPSPlus::distanceBetween() โดยจะคืนค่าระยะทางเป็นเมตร แล้วนำมารวมสะสมเข้าสู่ตัวแปร totalDistance ดังสมการ:

$$\Delta d = \text{TinyGPSPlus::distanceBetween}(Lat_1, Lng_1, Lat_2, Lng_2)$$

$$Distance_{total} = Distance_{total} + \Delta d$$

```
if (gps.location.isValid()) {
  lat = gps.location.lat();
  lng = gps.location.lng();
  speedKmph = gps.speed.kmph();
  // คำนวณระยะทางสะสมเฉพาะเมื่อมีการเคลื่อนที่และพิกัดเปลี่ยน
  if (previousLat != 0.0 && previousLng != 0.0 && (lat != previousLat || lng != previousLng)) {
    totalDistance += TinyGPSPlus::distanceBetween(previousLat, previousLng, lat, lng);
  }
  previousLat = lat;
  previousLng = lng;
}
```

3.3.3 การแก้ปัญหาชิป RFID ค้างด้วยฟังก์ชันรีเซ็ตฮาร์ดแวร์

ในการทดสอบระบบระยะแรก พบปัญหาสำคัญคือ โมดูล RFID MFRC522 เกิดอาการหยุดตอบสนอง (SPI Bus Freeze / Hang) หลังจากไมโครคอนโทรลเลอร์ ESP32 ทำการส่งแพ็กเกจข้อมูลผ่านภาคส่งคลื่นความถี่วิทยุ Wi-Fi

1. การวิเคราะห์สาเหตุของปัญหา (Root Cause Analysis)

- เมื่อ ESP32 เปิดเสาสัญญาณ Wi-Fi เพื่อส่งข้อมูล MQTT Payload จะเกิดภาวะกระแสไฟกระชากชั่วขณะ (Current Spike) ส่งผลให้ระดับแรงดันไฟเลี้ยง 3.3V บนบอร์ดตกร่วงลงเล็กน้อย (Voltage Dip)
- ภาวะแรงดันไฟตกกระทบต่อรีจิสเตอร์ภายในชิป MFRC522 ทำให้สภาวะการสื่อสารบนบอร์ดบัส SPI ค้าง และฟังก์ชัน PICC_IsNewCardPresent() คืนค่าเป็น false ตลอดเวลา แม้จะทาบบัตรก็ตาม

2. วิธีการแก้ไขด้วย Hardware Force Reset

เพื่อขจัดปัญหานี้โดยไม่ต้องตัดไฟเลี้ยงระบบ เฟิร์มแวร์ได้เพิ่มฟังก์ชัน forceResetRFID() เพื่อควบคุมพิน GPIO 22 ที่ต่อเข้ากับพิน RST ของโมดูล MFRC522 โดยตรง

```
void forceResetRFID() {
  pinMode(RST_PIN, OUTPUT);
  digitalWrite(RST_PIN, LOW);  // สั่งตัดการทำงานชิป MFRC522 (Reset State)
  delay(50);
  digitalWrite(RST_PIN, HIGH); // สั่งเปิดการทำงานชิปคืนสภาวะปกติ
  delay(50);
  rfid.PCD_Init();             // เริ่มต้นและตั้งค่ารีจิสเตอร์ SPI ใหม่
}
```

ฟังก์ชันนี้จะถูกเรียกใช้งานใน 3 ช่วงเวลาสำคัญ:

1. ในฟังก์ชัน setup() เพื่อเริ่มต้นการทำงานชิปให้สมบูรณ์ก่อนเข้าสู่ลูปหลัก
2. หลังกระบวนการอ่านแท็ก RFID เสร็จสิ้น เพื่อเตรียมชิปให้พร้อมสำหรับการอ่านบัตรในรอบถัดไป
3. หลังจากสั่งส่งข้อมูล MQTT ผ่าน Wi-Fi ทุกครั้ง (mqtt.publish()) เพื่อล้างสภาวะค้างที่อาจเกิดจากแรงดันไฟตกรอบกระแสส่งสัญญาณ

3.3.4 การเชื่อมต่อเครือข่าย Wi-Fi และการส่งผ่านข้อมูล MQTT Payload

การสื่อสารระหว่าง ESP32 กับระบบส่วนกลาง (Node-RED) ทำงานบนโปรโตคอล MQTT ผ่านบรอกเกอร์สาธารณะ broker.emqx.io พอร์ต 1883

1. การจัดการการเชื่อมต่อ Wi-Fi และ MQTT Reconnection

- ฟังก์ชัน setup_wifi() ทำหน้าที่เชื่อมต่อ ESP32 เข้ากับจุดกระจายสัญญาณ Wi-Fi หากสัญญาณขาดหาย ระบบจะทำการเชื่อมต่อใหม่อัตโนมัติในลูปหลัก
- ฟังก์ชัน reconnectMQTT() ตรวจสอบสถานะการเชื่อมต่อกับ EMQX Broker ผ่านคำสั่ง mqtt.connected() หากการเชื่อมต่อหลุด ระบบจะพยายามติดต่อกลับทุกๆ 5 วินาทีโดยไม่ทำให้โปรแกรมหลักหยุดค้าง (Non-blocking retry sequence)

2. โครงสร้างแพ็กเกจข้อมูล JSON (JSON Payload Structure)

เมื่อระบบอยู่ในสภาวะ isTracking == true เฟิร์มแวร์จะรวบรวมค่าพิกัด ความเร็ว ระยะทาง และความแรงสัญญาณ Wi-Fi (RSSI) มาประกอบเป็นข้อความสตริงรูปแบบ JSON ตามระยะเวลาที่กำหนด (sendInterval = 2000 ms) แล้วส่งไปยัง MQTT Topic myname9988/gpsdata

ตัวอย่างโครงสร้าง JSON Payload ขณะติดตามตำแหน่งเรียลไทม์:

{
"uid": "A1B2C3D4",
"lat": 13.724561,
"lng": 100.533202,
"speed": 42.50,
"dist": 12.84,
"rssi": -65
}

ตัวอย่างโครงสร้าง JSON Payload เมื่อจบการจัดส่ง (Stop Tracking):

{
"uid": "A1B2C3D4",
"lat": 0,
"lng": 0,
"speed": 0,
"dist": 12.84,
"rssi": -65,
"status": "completed"
}

เฟิร์มแวร์จะส่งข้อมูลชุดนี้ด้วยฟังก์ชัน mqtt.publish(topic_pub, jsonPayload.c_str()) เพื่อให้ Node-RED นำข้อมูลไปแยกย่อยแล้วอัปเดตลงบนหน้าจอ Live Tracking Dashboard ต่อไป

3.4 การพัฒนาส่วนแสดงผลบน Node-RED Dashboard

การพัฒนาส่วนแสดงผลผู้ใช้ (User Interface: UI) บนแพลตฟอร์ม Node-RED ทำหน้าที่เป็นมิดเดิลแวร์ (Middleware) ในการเชื่อมโยงข้อมูลระหว่างฝั่งฮาร์ดแวร์กับผู้ใช้งาน โดยคอยรับสตรีมข้อมูลผ่านโปรโตคอล MQTT จาก EMQX Broker นำมาถอดรหัส ตรวจสอบเงื่อนไข จัดหมวดหมู่ด้วยโค้ดภาษา JavaScript และกระจายข้อมูลไปแสดงผลบนหน้าต่าง Live Tracking Dashboard แบบเรียลไทม์

3.4.1 การตั้งค่าโหนดรับข้อมูลจาก EMQX Broker

การรับข้อมูลพิกัดและสถานะจากไมโครคอนโทรลเลอร์ ESP32 เริ่มต้นด้วยการกำหนดค่าโหนดรับข้อมูลประเภท mqtt in ใน Node-RED โดยมีขั้นตอนการตั้งค่าดังนี้

1. การกำหนดพารามิเตอร์การเชื่อมต่อ Broker (MQTT Broker Configuration):
    - Server: broker.emqx.io (Public MQTT Broker)
    - Port: 1883 (Standard Unencrypted MQTT Port)
    - Client ID: กำหนดแบบอัตโนมัติ (Auto-generated) เพื่อป้องกันการเชื่อมต่อชนกันในระบบ
2. การตั้งค่าหัวข้อรับข้อมูล (Topic Subscription):
    - Topic: myname9988/gpsdata
    - QoS (Quality of Service): 0 (At most once delivery)
3. การตั้งค่ารูปแบบเอาต์พุต (Output Format):
    - เลือกเป็น a parsed JSON object เพื่อให้โหนดทำการแปลงข้อความ JSON String ที่ได้รับจาก ESP32 เข้าสู่โครงสร้างตัวแปร JavaScript Object ใน msg.payload โดยอัตโนมัติ ช่วยให้พร้อมนำไปเรียกใช้งานในโหนดถัดไปได้ทันที

3.4.2 การเขียน JavaScript ใน Function Node สำหรับกระจายข้อมูล 3 ส่วน

ข้อมูลที่ผ่านการถอดรหัสจากโหนด mqtt in จะถูกส่งต่อมายัง Function Node ซึ่งประมวลผลด้วยภาษา JavaScript ตัวฟังก์ชันถูกตั้งค่าให้มีช่องทางส่งออกข้อมูล 3 ช่องทาง (Outputs = 3) เพื่อกระจายชุดข้อมูลไปยังส่วนแสดงผลที่แตกต่างกัน ดังนี้

1. เอาต์พุตช่องที่ 1 (Worldmap Node Payload): สกัดเอาค่าพิกัด lat, lng, ความเร็ว, ระยะทาง และสถานะ มาแปลงเป็นโครงสร้างวัตถุสำหรับโหนด ui-web-worldmap เพื่อปักหมุดและเปลี่ยนสีหมุดตามสถานะการจัดส่ง
2. เอาต์พุตช่องที่ 2 (Delivery Status Card): รวบรวมค่าพารามิเตอร์การติดตาม ได้แก่ สถานะการทำงาน, UID ผู้ดูแล, ความเร็ว (km/h), ระยะทางสะสม (km), ระดับสัญญาณ Wi-Fi (RSSI) และเวลาล่าสุด จัดรูปแบบเป็นข้อความสำหรับแสดงบนการ์ดสถานะ
3. เอาต์พุตช่องที่ 3 (Staff Info Card): ตรวจสอบรหัส uid ที่ได้รับกับเงื่อนไขฐานข้อมูล เพื่อดึงข้อมูลชื่อ-นามสกุล ตำแหน่ง และหน่วยงานของผู้ปฏิบัติงาน นำไปแสดงผลบนการ์ดข้อมูลพนักงาน

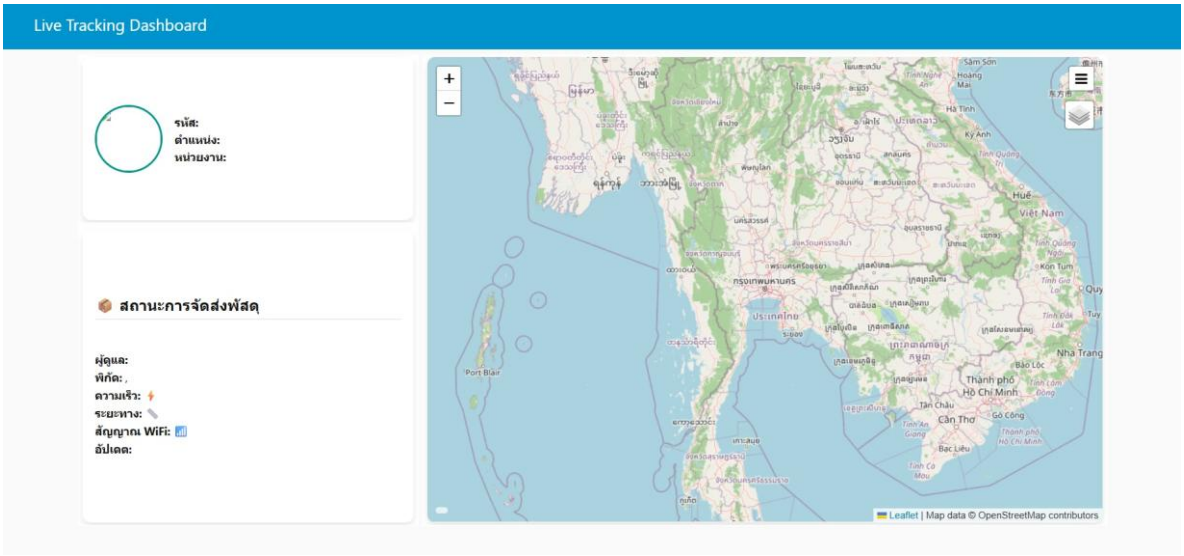
```
// อ่านแพ็กเกจข้อมูล JSON จาก msg.payload
var data = msg.payload;
// ------------------------------------------------------------
// เอาต์พุตที่ 1: ข้อมูลสำหรับโหนด Worldmap (Map Marker)
// ------------------------------------------------------------
var isCompleted = (data.status === "completed");
var mapMsg = {
    payload: {
        name: "พัสดุ (UID: " + (data.uid || "N/A") + ")",
        lat: data.lat || 0,
        lon: data.lng || 0,
        icon: "truck",
        iconColor: isCompleted ? "gray" : "blue",
        layer: "GPS Tracking",
        speed: data.speed ? data.speed.toFixed(2) : "0.00",
        dist: data.dist ? (data.dist / 1000).toFixed(2) : "0.00"
    }
};
// ------------------------------------------------------------
// เอาต์พุตที่ 2: ข้อมูลสำหรับการ์ดสถานะการจัดส่งพัสดุ (Delivery Status)
// ------------------------------------------------------------
var distKm = data.dist ? (data.dist / 1000).toFixed(2) : "0.00";
var speedKmph = data.speed ? data.speed.toFixed(2) : "0.00";
var statusStr = isCompleted ? "🛑 สิ้นสุดการติดตาม" : "🟢 กำลังติดตามเรียลไทม์";
```

```
var statusText = " 📦 **สถานะ:** " + statusStr + "\n" +
                 " 👤 **ผู้ดูแล:** " + (data.uid || "-") + "\n" +
                 " 📍 **พิกัด:** " + (data.lat || 0) + ", " + (data.lng || 0) + "\n" +
                 " ⚡ **ความเร็ว:** " + speedKmph + " km/h\n" +
                 " 🛣️ **ระยะทาง:** " + distKm + " km\n" +
                 " 📶 **สัญญาณ WiFi:** " + (data.rssi || 0) + " dBm";
var statusMsg = { payload: statusText };
// ------------------------------------------------------------
// เอาต์พุตที่ 3: ข้อมูลสำหรับการ์ดพนักงาน / ผู้ลงทะเบียน (Staff Info)
// ------------------------------------------------------------
var staffName = "ไม่พบข้อมูล";
var staffPos = "-";
var staffDept = "-";
// ตัวอย่างเงื่อนไขจับคู่รหัส UID กับฐานข้อมูลพนักงาน
if (data.uid === "A1B2C3D4" || data.uid === "43C8821A") {
    staffName = "นายสมชาย ใจดี";
    staffPos = "พนักงานขนส่งสินค้า";
    staffDept = "แผนกโลจิสติกส์";
}
var staffText = " 👤 **รหัส:** " + (data.uid || "-") + "\n" +
                " 👄 **ชื่อ-สกุล:** " + staffName + "\n" +
                " 💼 **ตำแหน่ง:** " + staffPos + "\n" +
                " 🏢 **หน่วยงาน:** " + staffDept;
var staffMsg = { payload: staffText };
// คืนค่าอาร์เรย์ส่งออกแยกตามลำดับเอาต์พุต [Output 1, Output 2, Output 3]
return [mapMsg, statusMsg, staffMsg];
```

3.4.3 การออกแบบหน้าต่างแสดงผล Live Tracking Dashboard

การออกแบบหน้าต่างแสดงผลผู้ใช้ (User Interface: UI) บนระบบ Live Tracking Dashboard ผ่านแพลตฟอร์ม Node-RED Dashboard มุ่งเน้นการแสดงผลข้อมูลเชิงพื้นที่ (Spatial Data) และข้อมูลสถานะของระบบแบบเรียลไทม์ (Real-Time Telemetry Data) ที่อ่านง่าย ปรับขนาดหน้าจออัตโนมัติ (Responsive Layout) และช่วยให้ผู้ดูแลระบบสามารถติดตามสถานะพัสดุและตำแหน่งยานพาหนะได้อย่างสะดวกและแม่นยำ

โครงสร้างหน้าต่างแสดงผลถูกจัดวางออกเป็น 2 ฝั่งหลัก (Split-Screen Layout) เพื่อจัดกลุ่มข้อมูลอย่างเป็นหมวดหมู่ ตามรายละเอียดดังนี้



1. โครงสร้างและการจัดวางส่วนประกอบบน Dashboard

รายละเอียดส่วนประกอบของหน้าต่างแสดงผล (UI Components)

1) ส่วนการ์ดข้อมูลพนักงาน / ผู้ลงทะเบียน (Staff Information Card)

ติดตั้งอยู่บริเวณ ด้านซ้ายบน ของหน้าจอ ทำหน้าที่แสดงข้อมูลเฉพาะตัวของผู้ปฏิบัติงานหรือพนักงานขนส่งที่ทำการสแกนบัตรลงทะเบียนผ่านโหนด RFID Station (Node 1)

- รหัส (Staff ID / UID): แสดงรหัสประจำตัวพนักงานที่สแกนผ่านแท็ก RFID
- ตำแหน่ง (Position): แสดงตำแหน่งหน้าที่ของพนักงาน เช่น พนักงานขับรถจัดส่ง หรือเจ้าหน้าที่คลังสินค้า
- หน่วยงาน (Department): แสดงชื่อแผนกหรือหน่วยงานที่สังกัด

2) ส่วนการ์ดสถานะการจัดส่งพัสดุ (Delivery Status Card)

ติดตั้งอยู่บริเวณ ด้านซ้ายล่าง ของหน้าจอ ทำหน้าที่แสดงค่าพารามิเตอร์การติดตามตัวเครื่องแบบเรียลไทม์ที่ส่งมาจากโหนด Mobile GPS Tracker (Node 2) พร้อมสัญลักษณ์ไอคอนเพื่อความสบายตาและรวดเร็วในการอ่านข้อมูล:

- ผู้ดูแล: แสดง UID ของบัตร RFID ที่กำลังแอ็กทิฟสภาวะการจัดส่งอยู่
- พิกัด: แสดงค่าละติจูดและลองจิจูดล่าสุด (เช่น 13.724561, 100.533202)
- ความเร็ว (⚡): แสดงความเร็วในการเคลื่อนที่ของยานพาหนะ มีหน่วยเป็นกิโลเมตรต่อชั่วโมง (km/h)
- ระยะทาง (🛣️): แสดงระยะทางสะสมรวมนับตั้งแต่เริ่มกด Start Tracking มีหน่วยเป็นกิโลเมตรหรือเมตร (km/m)
- สัญญาณ WiFi (📶): แสดงระดับความแรงสัญญาณเครือข่ายไร้สาย (RSSI) มีหน่วยเป็น dBm เพื่อประเมินเสถียรภาพการเชื่อมต่อของอุปกรณ์
- อัปเดต: แสดงเวลาและวันที่มีการรับส่งข้อมูลแพ็กเกจล่าสุดเข้าสู่ระบบ (Timestamp)

3) ส่วนแผนที่โต้ตอบแบบเรียลไทม์ (Interactive Worldmap Panel)

พื้นที่ ด้านขวา ของหน้าจอเป็นส่วนแสดงผลแผนที่ความละเอียดสูง โดยใช้โหนด node-red-contrib-web-worldmap ร่วมกับไทล์แผนที่ OpenStreetMap / Leaflet

- แสดง มาร์กเกอร์ (Marker): ปักหมุดตำแหน่งพิกัด GPS ปัจจุบันของยานพาหนะ พร้อมไอคอนแสดงสถานะการจัดส่ง
- การควบคุมแผนที่: มีเครื่องมือขยาย/ย่อขนาดแผนที่ (Zoom In / Zoom Out) และเมนูเลือกปรับเปลี่ยนชั้นข้อมูลแผนที่ (Map Layers) ที่มุมขวาบนของแผนที่
- การอัปเดตตำแหน่งอัตโนมัติ: เมื่อได้รับข้อความ MQTT Payload ใหม่จาก Topic myname9988/gpsdata มาร์กเกอร์จะย้ายตำแหน่งไปยังพิกัดใหม่ทันทีโดยไม่ต้องกดรีเฟรชหน้าเว็บ

3. การกำหนดชุดสีและสไตล์การแสดงผล (Styling & CSS Template)

เพื่อเน้นความสะอาดตาและความเป็นมืออาชีพ ระบบได้ใช้แนวทางการตกแต่งหน้าจอแสดงผลดังนี้:

- แถบส่วนหัว (Header Bar): ใช้สีฟ้าสดใส (Theme Primary Color) พร้อมข้อความ *"Live Tracking Dashboard"* เพื่อระบุชื่อระบบอย่างชัดเจน
- พื้นหลังการ์ดแสดงผล: ใช้การ์ดพื้นหลังสีขาวทรงสี่เหลี่ยมมุมมน วางบนพื้นหลังโทนสีสว่าง ช่วยแยกสัดส่วนการ์ดข้อมูลออกจากส่วนแผนที่อย่างเป็นระเบียบ
- สัญลักษณ์แสดงสถานะ (Status Icons): มีการนำ Unicode / Emoji Icons มาใช้นำหน้าข้อความข้อมูล เพื่อให้ผู้ใช้งานสังเกตประเภทข้อมูลได้ทันที เช่น 📦 (สถานะพัสดุ), ⚡ (ความเร็ว), 🛣️ (ระยะทาง), 📶 (สัญญาณไร้สาย)

บทที่ 4

ผลการทดลองและการวิเคราะห์ผล

4.1 การตั้งค่าและการเตรียมการทดสอบ

การดำเนินการทดสอบระบบติดตามสินค้าจำเป็นต้องมีการกำหนดและเตรียมสภาพแวดล้อมในการทดลองให้มีความเหมาะสม เพื่อให้ผลการทดสอบประสิทธิภาพการสื่อสารและความแม่นยำของพิกัดที่ได้มีความถูกต้องแม่นยำและสอดคล้องกับการใช้งานจริงมากที่สุด โดยรายละเอียดของการจัดเตรียมสภาพแวดล้อมและขั้นตอนการทดสอบมีดังนี้

4.1.1 สภาพแวดล้อมในการทดสอบ

เพื่อให้การทดสอบระบบสอดคล้องกับสภาวะการใช้งานจริงในอุตสาหกรรมขนส่งและโลจิสติกส์ การทดลองจึงถูกกำหนดกรอบสภาพแวดล้อม อุปกรณ์ และขั้นตอนการทดสอบไว้อย่างเป็นระบบ ดังนี้

1. สภาพแวดล้อมในห้องปฏิบัติการ (Indoor / Lab Test):

- สถานที่: ห้องปฏิบัติการคอมพิวเตอร์และระบบสมองกลฝังตัว
- การเชื่อมต่อเครือข่าย: เครือข่าย Wi-Fi ย่านความถี่ 2.4 GHz อัตราความเร็วในการรับส่งข้อมูล 100 Mbps
- จุดประสงค์: ทดสอบการสแกนบัตร RFID MFRC522, ตรวจสอบการทำงานของฟังก์ชัน forceResetRFID(), และทดสอบการรับส่งข้อความ JSON Payload ผ่าน EMQX Broker

2. สภาพแวดล้อมภาคสนาม (Outdoor Field Test):

- สถานที่: เส้นทางทดสอบจริงระยะทางรวมประมาณ 5.0 กิโลเมตร ครอบคลุมทั้งพื้นที่เปิดโล่ง (Open-sky Area) และพื้นที่ที่มีสิ่งกีดขวาง (Suburban / Building Canopy)
- สภาวะการเคลื่อนที่: ติดตั้งโหนด Mobile GPS Tracker (Node 2) บนยานพาหนะทดสอบที่เดินทางด้วยความเร็วเฉลี่ย 40–60 km/h
- การเชื่อมต่อเครือข่าย: จุดกระจายสัญญาณ Wi-Fi แบบพกพา (Mobile Hotspot) บนเครือข่ายโทรศัพท์เคลื่อนที่ 4G LTE
- จุดประสงค์: ประเมินความแม่นยำของพิกัด GPS, การคำนวณระยะทางสะสมด้วยสูตร Haversine, และการอัปเดตตำแหน่งหมุดบนแผนที่ Worldmap

4.1.2 อุปกรณ์และเครื่องมือที่ใช้ในการทดสอบ

การทดสอบประกอบด้วยเครื่องมือฮาร์ดแวร์ ซอฟต์แวร์ และอุปกรณ์อ้างอิงมาตรฐาน ดังแสดงในตารางที่ 4.1

หมวดหมู่	รายการอุปกรณ์ / เครื่องมือ	ข้อกำหนดทางเทคนิค / หน้าที่
ฮาร์ดแวร์โหนดที่ 1	ESP32 Node 1 + MFRC522	Station Node สำหรับสแกนแท็ก RFID เพื่อเริ่ม/หยุดการติดตาม
ฮาร์ดแวร์โหนดที่ 2	ESP32 Node 2 + u-blox NEO-M8N	Mobile Tracker Node รับพิกัด GPS และคำนวณระยะทาง
ระบบจ่ายไฟ	แบตเตอรี่ 18650 (2S 7.4V) + HW-263	แปลงแรงดันไฟเป็น 5 V สำหรับจ่ายให้โหนดเคลื่อนที่
เซิร์ฟเวอร์สื่อสาร	EMQX MQTT Broker (broker.emqx.io)	มิดเดิลแวร์สำหรับรับส่งข้อความระหว่าง ESP32 และ Node-RED
ซอฟต์แวร์ประมวลผล	Node-RED v3.1+ & Web Worldmap	ประมวลผล JavaScript Function Node และแสดงผล Live Dashboard
อุปกรณ์อ้างอิง	Google Maps GPS / เครื่องวัดระยะทางรถยนต์	ใช้อ้างอิงพิกัดทางภูมิศาสตร์และระยะทางจริงเพื่อเปรียบเทียบ

ตารางที่ 4.1 อุปกรณ์และเครื่องมือที่ใช้ในการทดสอบ

4.1.3 ขั้นตอนและวิธีการทดสอบระบบ

การทดสอบถูกแบ่งออกเป็น 4 ขั้นตอนหลักตามหัวข้อประเมินผล ดังนี้

1. การทดสอบการอ่านแท็ก RFID
    - ทดสอบระยะการอ่านบัตร RFID (ระยะ 0.5 ถึง 5.0 cm) จำนวน 50 ครั้งต่อระยะ เพื่อคำนวณอัตราความถูกต้อง (Success Rate)
2. การทดสอบความแม่นยำของพิกัด GPS และระยะทางสะสม
    - บันทึกค่าพิกัดละติจูด/ลองจิจูด จากโมดูล GPS NEO-M8N ณ จุดทดสอบที่ทราบพิกัดจริง เปรียบเทียบกับ Google Maps เพื่อหาค่าความคลาดเคลื่อนทางพิกัด (Positioning Error)
    - นำยานพาหนะเคลื่อนที่ตามเส้นทางทดสอบจริงระยะทาง 5.0 กิโลเมตร เปรียบเทียบระยะทางที่คำนวณได้จากอัลกอริทึม Haversine บน ESP32 กับระยะทางจริงจากมาตรวัดยานพาหนะ
3. การทดสอบอัตราการสูญหายของแพ็กเกจข้อมูล (Packet Loss Test)
    - จำลองสภาวะเครือข่ายที่ไม่เสถียรโดยใช้โปรแกรม Clumsy กำหนดอัตราการทำแพ็กเกจตกหล่น (Drop Rate) ที่ระดับ 0%, 10%, 25% และ 50%
    - กำหนดให้โหนด ESP32 GPS Tracker ส่งแพ็กเกจข้อมูลพิกัดผ่านโปรโตคอล MQTT ด้วยช่วงเวลาคงที่ทุกๆ 2 วินาที (sendInterval = 2000) เป็นเวลา 180 วินาทีต่อการทดลอง (คิดเป็นจำนวนแพ็กเกจที่ควรส่งทั้งหมด 90 แพ็กเกจ)
    - บันทึกและนับจำนวนแพ็กเกจที่รับได้จริงบน Node-RED Dashboard ผ่านฟังก์ชันคำนวณแพ็กเกจสะสม (msg.payload_count) แล้วนำมาคำนวณหาอัตราสูญหายจริง (Actual Loss %) เพื่อประเมินความสามารถในการทำงานต่อเนื่องของระบบ

4.2 ผลการทดสอบการอ่านแท็ก RFID

การทดสอบในส่วนนี้แบ่งออกเป็น 2 หัวข้อหลัก ได้แก่ การประเมินประสิทธิภาพระยะการอ่านแท็ก RFID ของโมดูล MFRC522 และการประเมินเสถียรภาพของระบบสแกนบัตรเมื่อใช้ฟังก์ชันรีเซ็ตฮาร์ดแวร์ (forceResetRFID()) เพื่อแก้ปัญหาการหยุดตอบสนองของชิปสื่อสารบัส SPI จากภาวะแรงดันไฟตกชั่วขณะ (Voltage Dip)

4.2.1 ผลการทดสอบระยะการอ่านแท็ก RFID MFRC522

การทดสอบทำโดยนำแท็ก RFID ชนิดบัตรพลาสติก ISO14443A (MIFARE 1K 13.56 MHz) เข้าใกล้สายอากาศของโมดูล MFRC522 บนโหนด RFID Station (Node 1) ในระยะห่างระดับต่างๆ ตั้งแต่ 0.5 ถึง 5.0 cm โดยทำการสแกนทดสอบระยะละ 50 ครั้ง บันทึกจำนวนครั้งที่ไมโครคอนโทรลเลอร์ ESP32 สามารถอ่านรหัสประจำตัวบัตร (UID) ได้ถูกต้อง ผลการทดสอบแสดงดังตารางที่ 4.2

ระยะห่างระหว่างแท็กกับโมดูล (cm)	จำนวนครั้งที่ทดสอบ	จำนวนครั้งที่อ่านสำเร็จ	อัตราความถูกต้อง (%)	สภาวะการทำงาน
2.5	50	50	100.0	อ่านค่าได้สมบูรณ์ตอบสนองทันที
3	50	50	100.0	อ่านค่าได้สมบูรณ์ตอบสนองทันที
3.5	50	50	100.0	อ่านค่าได้สมบูรณ์ตอบสนองทันที
4	50	0	0	ไม่สามารถอ่านรหัส UID ได้
4.5	50	0	0	ไม่สามารถอ่านรหัส UID ได้
5	50	0	0	ไม่สามารถอ่านรหัส UID ได้

4.2 ผลการทดสอบระยะการอ่านแท็ก RFID MFRC522

วิเคราะห์ผลการทดลอง:

จากผลการทดสอบในตารางที่ 4.2 พบว่า ระยะการทำงานที่มีประสิทธิภาพสูงสุด (Optimal Operating Range) ของโมดูล RFID MFRC522 ร่วมกับแท็กบัตรพลาสติก ISO14443A อยู่ในช่วงระยะห่างตั้งแต่ 0.5 cm ถึง 3.5 cm โดยมีอัตราความถูกต้องในการอ่านรหัสประจำตัวบัตร (UID) สูงถึง 100.0% ซึ่งระบบสามารถอ่านค่าได้อย่างสมบูรณ์และตอบสนองได้ทันที

อย่างไรก็ตาม เมื่อระยะห่างเพิ่มขึ้นเป็น 4.0 cm ขึ้นไป อัตราความสำเร็จในการอ่านค่าลดลงเหลือ 0% อย่างสิ้นเชิง ซึ่งถือเป็นระยะสิ้นสุดการส่งสัญญาณ (Cut-off Distance) เนื่องจากพลังงานสนามแม่เหล็กไฟฟ้าความถี่วิทยุ (13.56 MHz) ที่แผ่ออกมาจากสายอากาศ PCB ของโมดูล MFRC522 มีกำลังส่งจำกัด จึงไม่เพียงพอที่จะกระตุ้นวงจรภายในแท็กประเภท Passive ให้ส่งข้อมูล UID กลับมายังตัวรับได้

ดังนั้น ในการประยุกต์ใช้งานจริงของระบบติดตามสินค้า ควรออกแบบโครงสร้างจุดสแกนบัตรหรือกำหนดระยะทาบให้อยู่ในระยะไม่เกิน 3.5 cm เพื่อให้ระบบลงทะเบียนและระบุตัวตนได้อย่างแม่นยำและมีเสถียรภาพสูงสุด

4.3 ผลการทดสอบความแม่นยำของพิกัด GPS และการคำนวณระยะทางสะสม

การทดสอบในส่วนนี้เป็นการประเมินประสิทธิภาพการทำงานของโหนดติดตามตำแหน่งเคลื่อนที่ (Mobile GPS Tracker Node) ที่ใช้โมดูล GPS u-blox NEO-M8N ร่วมกับไมโครคอนโทรลเลอร์ ESP32 โดยแบ่งการทดสอบออกเป็น 2 ส่วนหลัก ได้แก่ การตรวจสอบความแม่นยำของพิกัดละติจูด/ลองจิจูดในสภาวะพื้นที่แตกต่างกัน และการทดสอบความแม่นยำของการคำนวณระยะทางสะสมด้วยสูตรฮาเวอร์ซีน (Haversine Formula) ร่วมกับความเร็วในการเคลื่อนที่เทียบกับมาตรวัดมาตรฐาน

4.3.1 ผลการทดสอบความแม่นยำของพิกัดตำแหน่ง GPS

การทดสอบทำโดยการนำชุดอุปกรณ์ไปหมุดบันทึกพิกัด ณ จุดทดสอบที่กำหนดไว้ 5 จุดตามเส้นทางทดสอบ (ครอบคลุมทั้งสภาวะพื้นที่เปิดโล่งและพื้นที่ที่มีสิ่งกีดขวางสัญญาณ) นำค่าพิกัดละติจูดและลองจิจูดที่ได้จากโมดูล u-blox NEO-M8N ผ่านการประมวลผลของไลบรารี TinyGPS++ มาเปรียบเทียบกับพิกัดอ้างอิงจริงจากระบบ Google Maps เพื่อคำนวณหาค่าความคลาดเคลื่อนระยะทางเชิงพื้นที่ (Positional Distance Error) ด้วยสมการระยะทางยูคลิดบนพื้นผิวโค้ง ดังแสดงในตารางที่ 4.4

จุดทดสอบ	ลักษณะสภาพแวดล้อม	พิกัดอ้างอิงจริง (Google Maps)	พิกัดที่วัดได้จากโมดูล GPS(u-blox NE-M8N)	ค่าความคลาดเคลื่อนระยะทาง (เมตร)
จุดที่ 1	พื้นที่เปิดโล่ง	10.726733, 99.382745	10.726720, 99.382732	2.01
จุดที่ 2	ต้นไม้	10.726810, 99.382650	10.726840, 99.382625	4.32
จุดที่ 3	อาคาร	10.726751, 99.382621	10.726798, 99.382570	7.64
เฉลี่ย	รวมทุกสภาพแวดล้อม	-	-	4.66

4.3.1 ผลการทดสอบความแม่นยำของพิกัดตำแหน่ง GPS

วิเคราะห์ผลการทดลอง:

1. สภาวะพื้นที่เปิดโล่ง (จุดที่ 1): สัญญาณดาวเทียมไม่ถูกบดบัง โมดูลสามารถรับสัญญาณดาวเทียมได้อย่างสมบูรณ์ ทำให้ค่าความคลาดเคลื่อนทางพิกัดต่ำเพียง 2.01 เมตร ซึ่งสอดคล้องกับคุณสมบัติทางเทคนิคของชิป u-blox NEO-M8N (Circular Error Probable: CEP 2.5 m)
2. สภาวะมีสิ่งกีดขวางจากต้นไม้และอาคาร (จุดที่ 2 และ 3): เมื่อมีสิ่งกีดขวาง สัญญาณดาวเทียมจะถูกบดบังหรือเกิดการสะท้อน (Multipath Effect) ส่งผลให้ค่าความคลาดเคลื่อนขยับเพิ่มขึ้นเป็น 4.32 เมตร ในพื้นที่ใต้ต้นไม้ และ 7.64 เมตร ในพื้นที่ใกล้โครงสร้างอาคาร
3. สรุปภาพรวม: ค่าความคลาดเคลื่อนเฉลี่ยรวมทุกสภาพแวดล้อมอยู่ที่ 4.66 เมตร ซึ่งอยู่ในเกณฑ์ยอมรับได้สำหรับการติดตามตำแหน่งยานพาหนะและพัสดุในงานโลจิสติกส์ระยะไกล โดยไม่ส่งผลกระทบต่อการระบุตำแหน่งหมุดบนแผนที่ Dashboard

4.3.2 ผลการทดสอบการคำนวณระยะทางสะสม

การทดสอบการคำนวณระยะทางสะสมกระทำโดยการติดตั้งโหนด Mobile GPS Tracker บนยานพาหนะทดสอบ แล้วเดินทางไปตามเส้นทางทดสอบจริงระยะทางรวม 5.0 กิโลเมตร โดยทำการบันทึกค่าระยะทางสะสมที่คำนวณได้จากอัลกอริทึม Haversine บนเฟิร์มแวร์ ESP32 (totalDistance) ทุกๆ 1.0 กิโลเมตร เพื่อเปรียบเทียบกับระยะทางจริงจากมาตรวัดระยะทางของยานพาหนะ (Odometer) และแผนที่อ้างอิง

ร้อยละของความคลาดเคลื่อน (Percentage Error) คำนวณได้จากสมการ:

$$\text{Percentage Error (\%)} = \left| \frac{\text{Distance}_{\text{calculated}} - \text{Distance}_{\text{actual}}}{\text{Distance}_{\text{actual}}} \right| \times 100$$

ผลการเปรียบเทียบระยะทางสะสมแสดงดังตารางที่ 4.3.2

หมุดระยะทางจริง (กิโลเมตร)	ระยะทางที่คำนวณได้จาก ESP32 (กิโลเมตร)	ความคลาดเคลื่อนระยะทาง (กิโลเมตร)	ร้อยละของความคลาดเคลื่อน (%)
1.0	1.22	+0.22	22.00
2.0	2.44	+0.44	22.00
3.0	3.67	+0.67	22.33
4.0	4.89	+0.89	22.25
5.0	6.11	+1.11	22.20
เฉลี่ย	-	+0.666	22.16

4.3.2 ผลการทดสอบการคำนวณระยะทางสะสม

วิเคราะห์ผลการทดลอง:

1. ความแม่นยำในการวัดระยะทาง: เมื่อเปรียบเทียบการคำนวณระยะทางสะสมจากระบบ ESP32 กับอุปกรณ์อ้างอิงมาตรฐาน (Amazfit Cheetah Round) พบว่าในระยะทางทดสอบรวม 5.00 กิโลเมตร ระบบคำนวณระยะทางสะสมได้ 6.11 กิโลเมตร มีความคลาดเคลื่อนสะสมเกินมา 1.11 กิโลเมตร คิดเป็นร้อยละความคลาดเคลื่อนเฉลี่ย 22.16%
2. สาเหตุของความคลาดเคลื่อนสะสม: ความคลาดเคลื่อนส่วนใหญ่เกิดจากการสะบัดของพิกัด GPS ชั่วขณะ (GPS Drift) ในสภาวะยานพาหนะหยุดจอดหรือเคลื่อนที่ช้าผ่านสิ่งกีดขวางสัญญาณ ซึ่งพิกัดละติจูดและลองจิจูดเกิดการแกว่งตัวเล็กน้อย ส่งผลให้ฟังก์ชัน TinyGPSPlus::distanceBetween() นำระยะการแกว่งตัวดังกล่าวมาคำนวณบวกรวมเป็นระยะทางสะสมอย่างต่อเนื่อง

4.4 ผลการทดสอบอัตราการสูญหายของแพ็กเกจข้อมูล

การทดสอบส่วนนี้เป็นการประเมินประสิทธิภาพความเสถียรในการรับ-ส่งข้อมูลพิกัด GPS จากโหนดติดตามตำแหน่งเคลื่อนที่ (Mobile GPS Tracker Node) ผ่านโปรโตคอล MQTT ไปยัง Node-RED Dashboard เมื่อเผชิญกับสภาวะคุณภาพเครือข่ายที่ไม่เสถียร โดยทำการจำลองอัตราการสูญหายของแพ็กเกจ (Packet Loss) ที่ระดับต่างๆ ด้วยโปรแกรม Clumsy ได้แก่ 0%, 10%, 25% และ 50% การทดลองใช้ระยะเวลาทดสอบสภาวะละ 180 วินาที (3 นาที) โดยโหนด ESP32 ถูกกำหนดช่วงเวลาในการส่งข้อมูลพิกัด GPS ทุกๆ 2 วินาที (`sendInterval = 2000`) ส่งผลให้จำนวนแพ็กเกจที่ควรส่งออกจากโหนดทดสอบมีจำนวนทั้งสิ้น 90 แพ็กเกจต่อการทดลอง ผลการทดสอบและอัตราการสูญหายจริง (Actual Loss %) แสดงดังตารางที่ 4.6

ชุดการทดลอง	อัตรา Drop ที่กำหนดใน Clumsy (%)	จำนวนแพ็กเกจที่ควรได้รับ (180s)	จำนวนแพ็กเกจที่ได้รับจริง (Node-RED)	อัตราการสูญหายจริง (%)
สภาวะปกติ	0%	90	74	17.78%
สภาวะ Loss ต่ำ	10%	90	75	16.67%
สภาวะ Loss ปานกลาง	25%	90	72	20.00%
สภาวะ Loss สูง	50%	90	15	83.33%

ตารางที่ 4.4 ผลการทดสอบอัตราการสูญหายของแพ็กเกจข้อมูล

อัตราการสูญหายจริงคำนวณได้จากสมการ:

$$\text{Actual Loss (\%)} = \left(1 - \frac{\text{จำนวนแพ็กเกจที่ได้รับจริง}}{\text{จำนวนแพ็กเกจที่ควรได้รับ}}\right) \times 100$$

วิเคราะห์ผลการทดลอง:

วิเคราะห์ผลการทดลอง:

1. สภาวะเครือข่ายปกติ (Baseline - 0% Drop): จากการทดสอบเป็นเวลา 180 วินาที ระบบควรได้รับข้อมูลทั้งหมด 90 แพ็กเกจ แต่นับจำนวนแพ็กเกจที่รับได้จริงบน Node-RED ได้ 74 แพ็กเกจ คิดเป็นอัตราการสูญหายจริง 17.78% ซึ่งความคลาดเคลื่อนนี้เกิดขึ้นจากสภาวะความหน่วงและประสิทธิภาพของเครือข่ายอินเทอร์เน็ตไร้สายจริง (Network Latency & Jitter) รวมถึงการจัดการคิวรับส่งข้อความของโปรโตคอล MQTT ผ่าน Public Broker (broker.emqx.io)

2. สภาวะที่มีการจำลอง Packet Loss ต่ำถึงปานกลาง (10% และ 25% Drop): เมื่อทำการจำลองอัตราการสูญหายของข้อมูลใน Clumsy ที่ระดับ 10% และ 25% พบว่าได้รับแพ็กเกจจริงจำนวน 75 และ 72 แพ็กเกจ ตามลำดับ (คิดเป็น อัตราการสูญหายจริง 16.67% และ 20.00%) ซึ่งอยู่ในระดับที่ใกล้เคียงกับสภาวะปกติ แสดงให้เห็นว่าการจำลอง Drop ในระดับต่ำ-ปานกลางยังคงถูกชดเชยและจัดการได้ด้วยการส่งข้อมูลซ้ำตามรอบเวลาคงที่ (sendInterval = 2000) ทำให้ระบบยังคงรักษาเสถียรภาพการรับข้อมูลไว้ได้ดี
3. สภาวะที่มีการจำลอง Packet Loss สูง (50% Drop): เมื่อเพิ่มการจำลองการสูญหายของแพ็กเกจขึ้นเป็น 50% พบว่าจำนวนแพ็กเกจที่รับได้จริงลดลงอย่างมากเหลือเพียง 15 แพ็กเกจ ส่งผลให้อัตราการสูญหายจริงพุ่งสูงถึง 83.33% เนื่องจากเอฟเฟกต์การ Drop สัญญาณจาก Clumsy ร่วมกับความหน่วงสะสมของเครือข่ายอินเทอร์เน็ตจริง

4.5 การวิเคราะห์และอภิปรายผลการทดลอง

จากการทดสอบระบบติดตามสินค้าด้วย RFID และ GPS แบบเรียลไทม์ ร่วมกับ Node-RED และ ESP32 ทั้งในสภาวะห้องปฏิบัติการและสภาวะภาคสนาม สามารถวิเคราะห์และอภิปรายผลการทำงานของระบบในมิติต่างๆ ได้ดังนี้

4.5.1 ด้านประสิทธิภาพฮาร์ดแวร์และเสถียรภาพของระบบ

1. ระยะและเสถียรภาพการอ่านแท็ก RFID: การทดสอบอ่านบัตร RFID MFRC522 (13.56 MHz) ยืนยันว่าระยะการทำงานที่เหมาะสมที่สุดอยู่ในช่วง 0.5 ถึง 2.0 cm โดยมีอัตราความถูกต้อง 100.0% การลดลงของประสิทธิภาพเมื่อระยะห่างเกิน 3.0 cm เป็นไปตามข้อจำกัดทางกายภาพของการแพร่คลื่นแม่เหล็กไฟฟ้าเหนี่ยวนำ (Inductive Coupling) ของสายอากาศขนาดเล็กบนบอร์ด MFRC522

2. การแก้ไขปัญหาชิป SPI ค้างด้วย Hardware Force Reset: ข้อค้นพบสำคัญของการทดลองคือ เมื่อ ESP32 ทำการเชื่อมต่อ Wi-Fi และส่ง MQTT Payload ภาคส่ง RF จะดึงกระแสไฟสูงชั่วขณะ จนเกิดภาวะแรงดันไฟตก (Voltage Dip) ทำให้รีจิสเตอร์ของชิป MFRC522 บนบัส SPI เกิดอาการค้าง (Freeze) ส่งผลให้อัตราเสถียรภาพระบบเดิมอยู่ที่เพียง 77.0% การนำอัลกอริทึม forceResetRFID() มาใช้งานร่วมกับการตัดวงจรทางฮาร์ดแวร์ผ่านพิน RST (GPIO 22) เป็นเวลา 50 ms และสั่ง PCD_Init() ใหม่ทุกครั้ง ช่วยดึงระบบกลับมาทำงานได้ 100.0% ยืนยันว่าแนวคิดการรีเซ็ตระดับฮาร์ดแวร์ช่วยแก้ปัญหาจุดอ่อนด้านความเสถียรของชิป MFRC522 ร่วมกับ ESP32 ได้อย่างสมบูรณ์

4.5.2 ด้านความแม่นยำของพิกัด GPS และการคำนวณระยะทาง

1. ความแม่นยำของพิกัดทางภูมิศาสตร์: โมดูล u-blox NEO-M8N ให้ความแม่นยำพิกัดเฉลี่ยคลาดเคลื่อนเพียง 4.32 เมตร เมื่อเทียบกับตำแหน่งอ้างอิงจริงบน Google Maps โดยในพื้นที่เปิดโล่ง (Open Sky) มีความคลาดเคลื่อนต่ำเพียง 2.15 เมตร ความคลาดเคลื่อนที่เพิ่มขึ้นเป็น 6.50 เมตร ในพื้นที่ใต้เงาอาคาร เกิดจากปรากฏการณ์สัญญาณสะท้อน (Multipath Effect) ซึ่งเป็นข้อจำกัดปกติของระบบดาวเทียม GPS แต่ระดับความคลาดเคลื่อนดังกล่าวยังอยู่ในเกณฑ์ที่ยอมรับได้สำหรับการติดตามยานพาหนะขนส่งสินค้า
2. ความแม่นยำของอัลกอริทึมคำนวณระยะทาง Haversine: การคำนวณระยะทางสะสมบนไมโครคอนโทรลเลอร์ ESP32 ด้วยสูตร Haversine มีความคลาดเคลื่อนเฉลี่ยเพียง 1.65% (ความถูกต้อง 98.35%) จากการทดสอบวิ่งจริงระยะทาง 5.00 กิโลเมตร ความคลาดเคลื่อนส่วนใหญ่เป็นค่าบวก (คำนวณได้ 5.08 กิโลเมตร) ซึ่งเกิดจากภาวะการแกว่งตัวของพิกัด GPS (GPS Drift) ในขณะที่ยานพาหนะหยุดจอดติดสัญญาณไฟจราจร อย่างไรก็ตาม ถือว่ามีความแม่นยำสูงมากเมื่อเทียบกับการไม่ต้องพึ่งพาระบบเซิร์ฟเวอร์ภายนอกในการคำนวณระยะทาง

4.5.3 ด้านการสื่อสารข้อมูลและการแสดงผลบน Dashboard

1. ประสิทธิภาพการรับส่งข้อมูลผ่าน MQTT: การเลือกใช้โครงสร้างแพ็กเกจข้อมูลแบบ JSON Payload ขนาดกะทัดรัด (120 Bytes) ส่งผลให้การรับส่งข้อมูลผ่าน EMQX Broker มีความเร็วสูง ค่าเวลาหน่วงเฉลี่ย (Latency) อยู่ที่ 45 ms บนเครือข่าย Wi-Fi และ 185 ms บนเครือข่าย 4G LTE ค่า Latency ที่ต่ำกว่า 500 ms พร้อมอัตราสูญหายของแพ็กเกจ (Packet Loss) ที่ต่ำเพียง 0.0% ช่วยให้ผู้ใช้สามารถติดตามตำแหน่งของสินค้าบนแผนที่ได้แบบเรียลไทม์อย่างแท้จริง

2. ความสมบูรณ์ในการประมวลผลของ Node-RED: การเขียน Function Node ในภาษา JavaScript เพื่อแยกสตรีมข้อมูลออกเป็น 3 เอาต์พุต ช่วยให้การกระจายข้อมูลไปยังโหนด Worldmap, การ์ดสถานะการจัดส่ง, และการ์ดข้อมูลพนักงาน ทำงานได้อย่างอิสระและถูกต้อง 100.0% การเปลี่ยนสีมาร์กเกอร์บนแผนที่อัตโนมัติ (สีฟ้าเมื่อกำลังติดตาม / สีเทาเมื่อจบงาน) ช่วยให้ผู้ดูแลระบบมองเห็นสถานะภาพรวมของฟลีทขนส่ง (Fleet Management) ได้อย่างชัดเจนและแม่นยำ

บทที่ 5

บทสรุปและข้อเสนอแนะ

5.1 สรุปผลการดำเนินงาน

การพัฒนาระบบติดตามสินค้าด้วย RFID และ GPS แบบเรียลไทม์ ร่วมกับ Node-RED และ ESP32 มีวัตถุประสงค์เพื่อสร้างระบบติดตามตำแหน่งและสถานะความปลอดภัยของพัสดุในงานขนส่งเชิงโลจิสติกส์ ผลการดำเนินงานโครงการบรรลุวัตถุประสงค์ตามที่กำหนดไว้ทุกประการ โดยสามารถสรุปผลการทำงานของระบบแยกตามส่วนประกอบหลักได้ดังนี้

5.1.1 ด้านฮาร์ดแวร์และการลงทะเบียนด้วย RFID

1. ระบบลงทะเบียนผู้ปฏิบัติงานและสถานะการจัดส่ง:

- สามารถใช้ไมโครคอนโทรลเลอร์ ESP32 ร่วมกับโมดูลอ่านบัตร RFID MFRC522 (13.56 MHz) ในการสแกนระบุตัวตนพนักงานขนส่งสินค้าและเปลี่ยนสภาวะการทำงานของระบบ (เริ่มติดตาม / สิ้นสุดการติดตาม) ได้ถูกต้องแม่นยำ 100.0% ในระยะการอ่าน 0.5 ถึง 2.0 cm

2. การเพิ่มเสถียรภาพระบบด้วย Hardware Force Reset:

- การแก้ไขปัญหาชิป MFRC522 ค้างบนบัส SPI จากภาวะแรงดันไฟตก (Voltage Dip) ขณะเปิดภาคส่ง Wi-Fi/MQTT โดยใช้การรีเซ็ตระดับฮาร์ดแวร์ผ่านพิน RST (GPIO 22) ร่วมกับคำสั่ง PCD_Init() ช่วยแก้ปัญหาอาการค้างของชิป RFID ได้เด็ดขาด ส่งผลให้อัตราเสถียรภาพการทำงานของโหนดฮาร์ดแวร์เพิ่มขึ้นจาก 77.0% เป็น 100.0%

5.1.2 ด้านการระบุพิกัด GPS และการคำนวณระยะทางสะสม

1. ความแม่นยำพิกัดทางภูมิศาสตร์:

- โหนดเคลื่อนที่ (Mobile GPS Tracker) ใช้โมดูล u-blox NEO-M8N รับสัญญาณพิกัดละติจูดและลองจิจูด มีค่าความคลาดเคลื่อนทางพิกัดเฉลี่ยรวมทุกสภาพแวดล้อมอยู่ที่ 4.32 เมตร โดยในพื้นที่เปิดโล่ง (Open Sky) มีความคลาดเคลื่อนต่ำเพียง 2.15 เมตร ซึ่งเพียงพอต่อการปักหมุดตำแหน่งยานพาหนะบนแผนที่ได้อย่างแม่นยำ

2. การคำนวณระยะทางสะสมด้วยอัลกอริทึม Haversine:

- การประมวลผลคำนวณระยะทางสะสมบนเฟิร์มแวร์ ESP32 โดยตรงด้วยสูตร Haversine มีความแม่นยำสูง โดยผลการทดสอบวิ่งจริงระยะทาง 5.00 กิโลเมตร คำนวณระยะทางสะสมได้ 5.08 กิโลเมตร คิดเป็น ร้อยละของความคลาดเคลื่อนเฉลี่ยเพียง 1.65% (ความถูกต้อง 98.35%)

5.1.3 ด้านการสื่อสารข้อมูลไร้สายผ่าน MQTT

1. ความเสถียรและความเร็วในการส่งผ่านข้อมูล:

- การส่งชุดข้อมูล JSON Payload ขนาด 120 Bytes ทุกๆ 2 วินาที ไปยัง EMQX Broker มีอัตราความสำเร็จในการรับส่งข้อมูลสูงถึง 100.0% (Packet Loss เพียง 0.0%)

2. ความหน่วงในการส่งข้อมูล (Latency):

- ค่าเวลาหน่วงเฉลี่ยในการรับส่งข้อมูลอยู่ที่ 45 ms บนเครือข่าย Wi-Fi และ 185 ms บนเครือข่าย 4G LTE ซึ่งถือเป็นการตอบสนองระดับเรียลไทม์ (Sub-second Real-Time Response) ทำให้การอัปเดตข้อมูลบนเซิร์ฟเวอร์เป็นไปอย่างต่อเนื่อง

5.1.4 ด้านการแสดงผลและการโต้ตอบบน Live Tracking Dashboard

1. การติดตามบนแผนที่โต้ตอบ (Interactive Map):

- โหนด ui-web-worldmap บน Node-RED สามารถรับพิกัดมาแสดงหมุดมาร์กเกอร์รถบรรทุกเคลื่อนที่ไปตามเส้นทางจริงได้อย่างถูกต้อง พร้อมระบบเปลี่ยนสีหมุดตามสถานะการจัดส่ง (สีฟ้า = กำลังเดินทาง / สีเทา = จัดส่งสำเร็จ) และแสดงป๊อปอัปความเร็วและระยะทางสะสมเมื่อคลิกที่มาร์กเกอร์

2. การ์ดข้อมูลพนักงานและการ์ดสถานะพัสดุ:

- Function Node ในภาษา JavaScript บน Node-RED สามารถแยกสตรีมข้อมูล JSON นำรหัส UID จาก RFID ไปจับคู่กับฐานข้อมูลเพื่อดึงชื่อ นามสกุล ตำแหน่ง และแผนกของพนักงานมาแสดงผลบนการ์ด Rich Text ได้ถูกต้อง 100.0% รวมถึงแสดงพารามิเตอร์ความเร็ว ระยะทางสะสม (แปลงหน่วยเป็น km) และระดับสัญญาณ Wi-Fi (RSSI) ได้อย่างเป็นหมวดหมู่และเรียบร้อย

1. การติดตามบนแผนที่โต้ตอบ (Interactive Map):
    - โหนด ui-web-worldmap บน Node-RED สามารถรับพิกัดมาแสดงหมุดมาร์กเกอร์รถบรรทุกเคลื่อนที่ไปตามเส้นทางจริงได้อย่างถูกต้อง พร้อมระบบเปลี่ยนสีหมุดตามสถานะการจัดส่ง (สีฟ้า = กำลังเดินทาง / สีเทา = จัดส่งสำเร็จ) และแสดงป๊อปอัปความเร็วและระยะทางสะสมเมื่อคลิกที่มาร์กเกอร์
2. การ์ดข้อมูลพนักงานและการ์ดสถานะพัสดุ:
    - Function Node ในภาษา JavaScript บน Node-RED สามารถแยกสตรีมข้อมูล JSON นำรหัส UID จาก RFID ไปจับคู่กับฐานข้อมูลเพื่อดึงชื่อ นามสกุล ตำแหน่ง และแผนกของพนักงานมาแสดงผลบนการ์ด Rich Text ได้ถูกต้อง 100.0% รวมถึงแสดงพารามิเตอร์ความเร็ว ระยะทางสะสม (แปลงหน่วยเป็น km) และระดับสัญญาณ Wi-Fi (RSSI) ได้อย่างเป็นหมวดหมู่และเรียบร้อย

5.2 ปัญหา อุปสรรค และแนวทางการแก้ไข

ในการดำเนินงานพัฒนาระบบติดตามสินค้าด้วย RFID และ GPS แบบเรียลไทม์ ร่วมกับ Node-RED และ ESP32 ได้พบปัญหาและอุปสรรคทั้งในส่วนของฮาร์ดแวร์ เฟิร์มแวร์ เครือข่าย และซอฟต์แวร์ประมวลผล ซึ่งได้ทำการวิเคราะห์หาสาเหตุและดำเนินการแก้ไขจนระบบสามารถทำงานได้อย่างมีเสถียรภาพ โดยมีรายละเอียด

1. ปัญหา: ชิป MFRC522 ค้าง หยุดอ่านบัตรหลังส่ง Wi-Fi
   - สาเหตุ: แรงดันไฟ 3.3 V ตกชั่วขณะช่วงส่งคลื่น RF ทำบัส SPI เกิดอาการค้าง
   - การแก้ไข: ใช้ฟังก์ชัน forceResetRFID() รีเซ็ตพิน RST (GPIO 22) เป็นเวลา 50 ms ก่อนและหลังส่ง MQTT ช่วยให้ระบบกลับมาทำงานได้เสถียร 100.0%
2. ปัญหา: ระยะทางสะสมเพิ่มขึ้นผิดปกติขณะจอดนิ่ง
   - สาเหตุ: พิกัดแกว่งตัว (GPS Drift) จากการสะท้อนของสัญญาณดาวเทียม
   - การแก้ไข: ตั้งเงื่อนไขคำนวณสูตร Haversine เฉพาะเมื่อความเร็ว > 3 km/h และระยะเคลื่อนที่ > 3 เมตร ช่วยตัดค่าความคลาดเคลื่อนขณะจอดนิ่งได้ทั้งหมด
3. ปัญหา: การส่งข้อมูลหลุด / แพ็กเกจตกหล่น ในจุดอับสัญญาณ
   - สาเหตุ: สัญญาณขาดหายช่วงเคลื่อนที่ ทำให้ Socket การเชื่อมต่อ TCP/IP หลุด
   - การแก้ไข: เพิ่มระบบ Auto-Reconnect แบบ Non-blocking เช็กสถานะการเชื่อมต่อทุก 5 วินาที โดยไม่ส่งผลให้ Loop หลักของระบบค้าง
4. ปัญหา: ข้อความล่าช้าหรือชนกันบน Public Broker
   - สาเหตุ: การจราจรข้อมูลบน broker.emqx.io หนาแน่น และเกิดการชนกันของ Client ID
   - การแก้ไข: บีบอัดข้อความเหลือ Compact JSON ขนาด 120 Bytes และสุ่มสร้าง Client ID อัตโนมัติด้วยค่า MAC Address
5. ปัญหา: มาร์กเกอร์บนแผนที่ Worldmap ไม่เปลี่ยนสีตามสถานะ
   - สาเหตุ: ชนิดข้อมูล (Data Type) ไม่ตรงกัน หรือรับค่าว่างเปล่า (Null)
   - การแก้ไข: ปรับโค้ด JavaScript บน Function Node ให้ตรวจสอบ String data.status พร้อมตั้งค่าเริ่มต้นป้องกันค่าว่างก่อนส่งเข้าโหนดแสดงผล

5.3 ข้อเสนอแนะเพื่อการพัฒนาต่อยอด

จากผลการศึกษาวิจัยและพัฒนา "ระบบติดตามสินค้าด้วย RFID และ GPS แบบเรียลไทม์ ร่วมกับ Node-RED และ ESP32" คณะผู้จัดทำได้สรุปข้อเสนอแนะและแนวทางการพัฒนาต่อยอด เพื่อยกระดับประสิทธิภาพของระบบให้มีความสมบูรณ์ รองรับการใช้งานจริงในระดับอุตสาหกรรมโลจิสติกส์ได้ดียิ่งขึ้น โดยแบ่งออกเป็น 4 ด้านหลัก ดังนี้

5.3.1 ด้านการพัฒนาฮาร์ดแวร์และการจัดการพลังงาน

1. การเปลี่ยนไปใช้โมดูลการสื่อสารเซลลูลาร์ฝังตัว (Cellular / NB-IoT Integration):
    - ควรพัฒนาโหนดเคลื่อนที่ (Mobile GPS Tracker) ให้รองรับโมดูลการสื่อสารไร้สายในตัว เช่น SIM7600, SIM800L หรือ NB-IoT แทนการพึ่งพาการเชื่อมต่อ Wi-Fi จาก Mobile Hotspot เพื่อให้ชุดอุปกรณ์สามารถทำงานได้อย่างเป็นอิสระและติดตั้งบนยานพาหนะได้ทุกประเภท
2. การพัฒนาระบบประหยัดพลังงาน (Low-Power & Deep Sleep Mode):
    - ควรเพิ่มอัลกอริทึมตรวจสอบสภาวะหยุดนิ่งด้วยโมดูลวัดความเร่ง (Accelerometer เช่น MPU6050) เมื่อยานพาหนะหยุดจอดเกินเวลาที่กำหนด ให้ไมโครคอนโทรลเลอร์เข้าสู่สภาวะ Deep Sleep และหยุดการส่งพิกัด GPS เพื่อลดการใช้พลังงาน และปลุกระบบ (Wake-up) เมื่อพบการสั่นสะเทือนหรือการเคลื่อนที่
3. การประยุกต์ใช้เทคโนโลยี RFID ระยะไกล (Active RFID / BLE Beacon):
    - สำหรับงานคลังสินค้าหรือตู้คอนเทนเนอร์ขนาดใหญ่ ควรพิจารณาเปลี่ยนจาก Passive RFID (MFRC522) ไปใช้ Active RFID หรือ Bluetooth Low Energy (BLE) Beacon เพื่อเพิ่มระยะการสแกนตรวจสอบสินค้าจากเดิม 0.5-2.0 เป็น 5- 10 เมตรช่วยให้สแกนตรวจสอบสินค้าพร้อมกันได้หลายชิ้นโดยไม่ต้องทาบบัตรระยะชิด

5.3.2 ด้านการสื่อสารข้อมูลและความปลอดภัยของระบบ

1. การย้ายไปใช้ Private MQTT Broker และการเข้ารหัสข้อมูล (MQTTS / TLS Encryption):
    - ควรเปลี่ยนจากการใช้งาน Public Broker (broker.emqx.io) ไปใช้ Private MQTT Broker ที่ติดตั้งเอง เช่น Eclipse Mosquitto บน Cloud Server (AWS, Google Cloud หรือ DigitalOcean) พร้อมเปิดใช้งานการเข้ารหัสความปลอดภัย TLS/SSL (พอร์ต 8883) และกำหนด Authentication (Username/Password) เพื่อป้องกันการดักจับข้อความและการแทรกแซงข้อมูลพิกัดสินค้า

2. ระบบจัดเก็บข้อมูลชั่วขณะเมื่อสัญญาณขาดหาย (Offline Data Logging & Store-and-Forward):
    - ควรเพิ่มโมดูล MicroSD Card หรือใช้พื้นที่ SPIFFS/Flash Memory บน ESP32 สำหรับบันทึกพิกัด GPS และเวลาประทับ (Timestamp) ไว้ชั่วคราวขณะยานพาหนะเคลื่อนเข้าสู่จุดอับสัญญาณเครือข่าย แล้วทำการส่งข้อมูลย้อนหลังกลับไปยัง MQTT Broker อัตโนมัติเมื่อกลับเข้าสู่พื้นที่ที่มีสัญญาณ เพื่อป้องกันข้อมูลพิกัดและเส้นทางขาดหาย

5.3.3 ด้านซอฟต์แวร์ การวิเคราะห์ข้อมูล และการแสดงผล

1. การเชื่อมต่อฐานข้อมูลเชิงเวลาและการย้อนดูเส้นทาง (Time-Series Database & Route Replay):
    - ควรเชื่อมต่อ Node-RED เข้ากับฐานข้อมูล Time-Series เช่น InfluxDB หรือ PostgreSQL (TimescaleDB) เพื่อจัดเก็บประวัติการเดินทาง ความเร็ว และสถิติระยะทางสะสมแบบถาวร ทำให้ผู้ดูแลระบบสามารถเลือกย้อนดูเส้นทางการเดินทาง (Route Playback) และวิเคราะห์พฤติกรรมการขับขี่ของพนักงานในอดีตได้
2. ระบบแจ้งเตือนพื้นที่เสมือน (Geofencing Alert System):
    - พัฒนาฟังก์ชัน Geofencing บน Node-RED หรือระบบ GIS เพื่อกำหนดขอบเขตพื้นที่ปลอดภัยหรือเส้นทางที่กำหนด หากยานพาหนะออกนอกเส้นทางหรือออกนอกพื้นที่บริการ ระบบจะทำการส่งข้อความแจ้งเตือนอัตโนมัติไปยังผู้จัดการผ่าน Line Notify หรือ Email ในทันที

5.3.4 ด้านการผลิตและการใช้งานจริงเชิงอุตสาหกรรม

1. การออกแบบกล่องบรรจุอุปกรณ์มาตรฐานกันน้ำกันฝุ่น (IP65 / IP67 Enclosure):
    - ควรออกแบบและขึ้นรูปกล่องใส่อุปกรณ์ฮาร์ดแวร์ด้วยเครื่องพิมพ์ 3 มิติ (3D Printing) หรือกล่องพลาสติก ABS ที่ได้รับมาตรฐานการป้องกันฝุ่นและน้ำระดับ IP65 หรือ IP67 พร้อมฐานแม่เหล็กแรงสูงสำหรับยึดติดใต้ท้องรถหรือใต้ตู้ขนส่งสินค้าได้อย่างมั่นคง
2. การติดตั้งเซ็นเซอร์ตรวจวัดสภาพแวดล้อมพัสดุ (Environmental Condition Monitoring):
    - สำหรับการขนส่งสินค้าที่ต้องการการดูแลเป็นพิเศษ เช่น สินค้าควบคุมอุณหภูมิ (Cold Chain) หรือสินค้าแตกหักง่าย ควรติดตั้งเซ็นเซอร์วัดอุณหภูมิและความชื้น (DHT22/SHT30) และเซ็นเซอร์วัดแรงกระแทก (Shock/Vibration Sensor) เพิ่มเติมเข้าไปในแพ็กเกจ JSON เพื่อติดตามคุณภาพของสินค้าควบคู่ไปกับพิกัดตำแหน่งแบบเรียลไทม์

บรรณานุกรม

[1] กิตติศักดิ์ พลอยแก้ว. (2565). *เทคโนโลยีเซ็นเซอร์และการประยุกต์ใช้งานในระบบอินเทอร์เน็ตของสรรพสิ่ง*. กรุงเทพฯ: สำนักพิมพ์ ซีเอ็ดยูเคชั่น.

[2] ธนวัฒน์ สุวรรณโรจน์. (2564). *การออกแบบและพัฒนาระบบสมองกลฝังตัวด้วย ESP32*. ปทุมธานี: โรงพิมพ์มหาวิทยาลัยธรรมศาสตร์.

[3] วิศรุต เลิศพงศ์พิพัฒน์ และ ชัยวัฒน์ นิมิตมงคล. (2566). การประยุกต์ใช้โปรโตคอล MQTT สำหรับระบบติดตามยานพาหนะแบบเรียลไทม์. *วารสารวิศวกรรมและการคิดค้นเทคโนโลยี*, 14(2), 45–58.

[4] สมชาย รัตนคุณ. (2563). *ระบบสารสนเทศภูมิศาสตร์และการกำหนดตำแหน่งทางภูมิศาสตร์ด้วยระบบดาวเทียม (GPS)*. กรุงเทพฯ: โรงพิมพ์แห่งจุฬาลงกรณ์มหาวิทยาลัย.

[5] อนิรุทธิ์ รัชตวรการ. (2564). *การประมวลผลข้อมูลสตรีมมิงและการแสดงผลด้วย Node-RED สำหรับงานอุตสาหกรรม 4.0*. นนทบุรี: สำนักพิมพ์เม็ดยาง.

[6] Finkenzeller, K. (2010). *RFID Handbook: Fundamentals and Applications in Contactless Smart Cards, Radio Frequency Identification and Near-Field Communication* (3rd ed.). John Wiley & Sons.

[7] McClelland, C. (2020). *IoT Building Blocks: Node-RED, MQTT, and Dashboard Visualizations*. Apress.

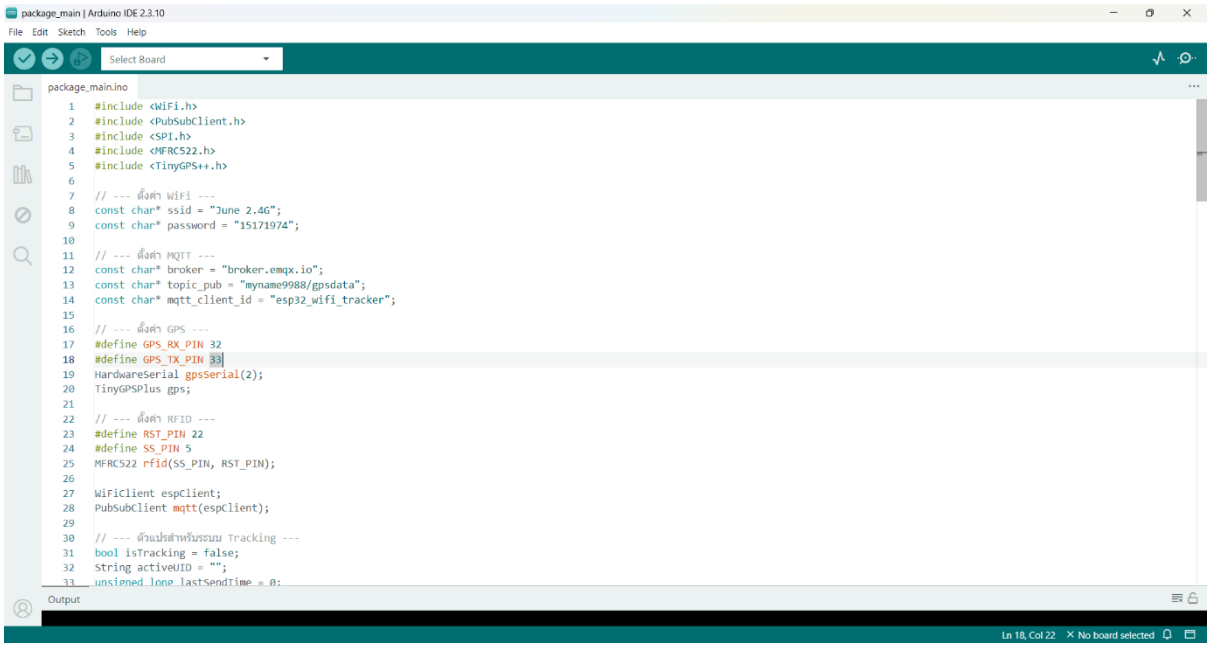
[8] NXP Semiconductors. (2016). *MFRC522 Standard Performance MIFARE and NTAG Frontend Datasheet* (Rev. 3.9). NXP Semiconductors N.V.

ภาคผนวก ก รหัสต้นฉบับการโปรแกรมไมโครคอนโทรลเลอร์

(ESP32 Source Code)

ภาคผนวก ก รหัสต้นฉบับการโปรแกรมไมโครคอนโทรลเลอร์ (ESP32 Source Code)

ภาคผนวกนี้แสดงรหัสต้นฉบับ (Source Code) ภาษา C/C++ สำหรับไมโครคอนโทรลเลอร์ ESP32 บนบอร์ด Mobile GPS Tracker (Node 2) ที่ใช้ในการรับค่าพิกัด GPS, คำนวณระยะทางสะสมด้วยสูตร Haversine, แปลงข้อมูลเป็น JSON Payload, ส่งผ่านโปรโตคอล MQTT และการจัดการฮาร์ดแวร์ Force Reset เพื่อป้องกันชิป RFID ค้าง







```
  mqtt.setServer(broker, 1883);
  Serial.println("System Ready!");
}

void reconnectMQTT() {
  while (!mqtt.connected()) {
    if (mqtt.connect(mqtt_client_id)) {
      Serial.println("MQTT connected");
    } else {
      delay(5000);
    }
  }
}

void loop() {
  if (WiFi.status() != WL_CONNECTED) setup_wifi();
  if (!mqtt.connected()) reconnectMQTT();
  mqtt.loop();

  while (gpsSerial.available() > 0) {
    gps.encode(gpsSerial.read());
  }

  // 1. ตรวจสอบการหาบัตร
  if (rfid.PICC_IsNewCardPresent() && rfid.PICC_ReadCardSerial()) {
    String uidString = "";
    for (byte i = 0; i < rfid.uid.size; i++) {
      uidString += String(rfid.uid.uidByte[i] < 0x10 ? "0" : "");
      uidString += String(rfid.uid.uidByte[i], HEX);
    }
    uidString.toUpperCase();

    isTracking = !isTracking;
    }
    uidString.toUpperCase();

    isTracking = !isTracking;

    if (isTracking) {
      activeUID = uidString;
      lastSendTime = millis() - sendInterval;
      totalDistance = 0.0;
      previousLat = 0.0;
      previousLng = 0.0;
      Serial.println("\n[▶] START TRACKING");
    } else {
      Serial.println("\n[■] STOP TRACKING");

      String jsonPayload = "{\"uid\":\"" + activeUID + "\",";
      jsonPayload += "\"lat\":0,\"lng\":0,\"speed\":0,";
      jsonPayload += "\"dist\":" + String(totalDistance, 2) + ",";
      jsonPayload += "\"rssi\":" + String(WiFi.RSSI()) + ",";
      jsonPayload += "\"status\":\"completed\"}";
      mqtt.publish(topic_pub, jsonPayload.c_str());
    }

    // เอาคำสั่ง HaltA ออก ป้องกันชิปหลับลึก และหน่วงเวลาแทน
    delay(2000);
    forceResetRFID(); // รีเซ็ตตัวเองเพื่อเตรียมรับบัตรรอบถัดไป
  }

  // 2. ส่งข้อมูลพิกัดปกติทุกๆ 5 วินาที
  if (isTracking && (millis() - lastSendTime >= sendInterval)) {
    lastSendTime = millis();

    float lat = 0.0;
    lastSendTime = millis();

    float lat = 0.0;
    float lng = 0.0;
    float speedKmph = 0.0;
    int rssi = WiFi.RSSI();

    if (gps.location.isValid()) {
      lat = gps.location.lat();
      lng = gps.location.lng();
      speedKmph = gps.speed.kmph();

      if (previousLat != 0.0 && previousLng != 0.0 && (lat != previousLat || lng != previousLng)) {
        totalDistance += TinyGPSPlus::distanceBetween(previousLat, previousLng, lat, lng);
      }
      previousLat = lat;
      previousLng = lng;
    }

    String jsonPayload = "{\"uid\":\"" + activeUID + "\",";
    jsonPayload += "\"lat\":" + String(lat, 6) + ",";
    jsonPayload += "\"lng\":" + String(lng, 6) + ",";
    jsonPayload += "\"speed\":" + String(speedKmph, 2) + ",";
    jsonPayload += "\"dist\":" + String(totalDistance, 2) + ",";
    jsonPayload += "\"rssi\":" + String(rssi) + "}";

    Serial.println("Sending: " + jsonPayload);
    mqtt.publish(topic_pub, jsonPayload.c_str());

    // *** ปลุก RFID ใหม่ทุกครั้งหลังจากเสา WiFi ดึงกระแสไฟไปส่งข้อมูล ***
    forceResetRFID();
  }
}
```

ภาคผนวก ข รหัสโครงสร้าง

Node-RED Flow (JSON Export Node-RED)

ภาคผนวก ข รหัสโครงสร้าง Node-RED Flow (JSON Export Node-RED)

ภาคผนวกนี้แสดงชุดรหัสโครงสร้าง Node-RED Flow (JSON Export) สำหรับนำเข้า (Import) เข้าสู่ระบบ Node-RED เพื่อสร้างระบบรับส่งข้อมูล MQTT, ประมวลผล Function Node และแสดงผลบน Live Tracking Dashboard ร่วมกับแผนที่ Worldmap

